

Contents

1	Einführung	3
1.1	Software	3
1.2	Software Engineering	4
2	Agiles Software Engineering	7
2.1	Motive und Prinzipien	7
2.2	Scrum als agiles Prozessmodell	8
2.2.1	Überblick	8
2.2.2	Scrum Planung	9
2.2.3	Durchführung von Sprints	10
2.3	Fragen	11
3	Analyse	13
3.1	Anforderungen	13
4	Weiteres	15
4.0.1	Fragen	16
5	Fragen	19
5.0.1	Planung bei Scrum	19
5.0.2	Charakterisierung Software-Engineering	21
5.0.3	Herausforderungen bei der Anforderungsanalyse	22
5.0.4	Struktur eines Anwendungsfalls	24
5.0.5	Agile Softwareentwicklung	24
5.0.6	Kopplung von Klassen	27
5.0.7	GRASP	28
5.0.8	CRC-Kartes	30
5.0.9	High Cohesion	31
5.0.10	Black-Box-Tests (BBTs)	31
5.0.11	Glass-Box-Tests (GBTs)	31
5.0.12	Domänenmodell	34
5.0.13	Qualitätsmerkmale von Software	34
5.0.14	Scrum-Team, Schätzungen und Planning Poker	37
5.0.15	Velocity	38
5.0.16	Model-View-Controller (MVC)	38
5.0.17	Prozessmodell in der Software-Entwicklung	39
5.0.18	Testfälle für Fakultätsfunktion	40
5.0.19	Aufwandsschätzung mit Use Case Pointss (UCPs)	40
5.0.20	Äquivalenzklassen	41
5.0.21	Vorgangsknotennetz	42
5.0.22	Erfolg eines IT-Projekts	43
5.0.23	Entwurfsmuster	44
5.0.24	Unterschied Architektur- und Entwurfsmuster	45
5.0.25	Architekturmuster	45
6	Anhang	47

Chapter 1

Einführung

1.1 Software

Software

Die Programme, Verfahren, zugehörige Dokumentation und Daten, die mit dem Betrieb eines Rechnersystems zu tun haben. (IEEE 610.12)

Typen von Software

- **Individualsoftware:**
 - Software mit spezifischer Funktionalität, die für Auftraggeber*innen entwickelt wird
- **Softwareprodukte:**
 - Softwareprodukte werden für einen Markt mit einem bestimmten Bedarf entwickelt
 - Normalerweise keine direkten Auftraggeber*innen
- **Softwareplattform:**
 - Ein Softwareprodukt, das den Aufbau neuer Anwendungen in derselben Plattform erlaubt
- **Software realisiert Systeme:**
 - Mengen von Programmen genügen nicht; wir brauchen Systeme
 - Zusammenspiel vieler verteilter Bausteine aus Hardware und Software notwendig
 - Durch die (notwendige) Systemintegration wird aus vielen einfachen kleinen Teilproblemen ein komplexes und schwieriges Gesamtproblem!

Eigenschaften von Software

- **immateriell:** Software ist abstrakt und mit unseren Sinnen nicht direkt wahrnehmbar!
 - Fehler sind schwieriger erkennbar
 - Entwicklungsstand sowie Qualität schwer zu beurteilen
- **unstet**
 - Kleinste Veränderungen der Software führen ggf. zu massiven Änderungen im Systemverhalten
 - Zusammenhang zwischen Eingabe und Ausgabe nicht durch eine kontinuierliche Funktion beschreibbar
 - Valider Funktionsnachweis schwierig, insbesondere im Nachhinein, ggf. schwieriger als in anderen Domänen
 - Nicht-triviale Softwaresysteme sind intrinsisch komplex
- **benötigt Qualität**



Figure 1.1: Caption

- Zuverlässigkeit: Software darf im Fall des Versagens keine physischen oder ökonomischen Schäden verursachen
- Benutzbarkeit: Software muss sich nach den Bedürfnissen der Benutzer richten, die Benutzerschnittstelle muss ergonomisch und selbsterklärend sein und die Dokumentation muss zielgerecht zur Verfügung stehen
- Wartbarkeit: Software muss an neue Anforderungen anpassbar sein und sollte möglichst plattformunabhängig sein
- Effizienz: Software muss ökonomischen Gebrauch von Ressourcen des unterliegenden Systems machen
- nicht-funktionale Qualitätseigenschaften ??

- **verändert Realität**

- Automatisierung oder Unterstützung von menschlicher Arbeit, technischen oder sozialen Abläufen und in beliebigen Anwendungsbereichen
- in ständiger Wechselwirkung mit Arbeitsprozessen, Produktionsprozessen und Menschen
- hat zwei hauptsächliche **Herausforderungen**: Das Problem im Kontext des jeweiligen Anwendungsbereichs verstehen und die Problemlösung auf Software abbilden

Der Software Lebenszyklus

- **Analyse und Spezifikation**: Was soll das System machen?
- **Entwicklung**: Erstellung des Systems, Integration und Test.
- **Abnahme**: Übereinstimmung mit den Kundenanforderungen.
- **Betrieb**: Installation beim Kunden, Wartung.
- **Evolution**: Weiterentwicklung gemäß neuer Kundenanforderungen oder Ausmusterung.

1.2 Software Engineering

Software Engineering

Software Engineering ist die Anwendung eines **systematischen, disziplinierten** und **quantifizierbaren** Ansatzes auf die **Entwicklung**, den **Betrieb** und die **Wartung** von Software, das heißt, die Anwendung der Prinzipien des Ingenieurwesens auf Software. (IEEE 610.12)

- **Systematisch**: Klar abgegrenzte Einzelschritte und Ein- und Ausgaben sind definiert
- **Diszipliniert**: Vorgehen folgt organisierten Prinzipien, feste Rollendefinitionen, sorgfältige Dokumentation
- **Quantifizierbar**: Schätzung und Messung von relevanten Größen und vorhersagbar mit begründbaren und akzeptablen Grenzen

Typen von Software Engineering

Merkmal	Projektbasiertes Software Engineering	Produktorientiertes Software Engineering	Produktmanagement in Softwareprojekten
Ausgangspunkt	Kundenanforderungen	Geschäftsidee oder Marktlücke	Interne strategische Planung
Zielsetzung	Individuelle Software, die spezifische Anforderungen erfüllt	Entwicklung eines marktfähigen Softwareprodukts	Steuerung der Entwicklung, Vermarktung und Wartung eines Produkts
Kundenfokus	Externer Kunde mit spezifischen Anforderungen	Kein externer Kunde; Fokus auf Marktanforderungen	Schnittstelle zwischen Kunden, Unternehmen und Entwicklungsteam
Flexibilität	Anforderungen können sich während der Entwicklung ändern	Time-to-Market hat Priorität, daher weniger flexibel	Anpassung an Kundenbedürfnisse und Marktanforderungen
Lebensdauer der Software	Lange Lebensdauer (10 Jahre oder mehr) mit langfristiger Unterstützung	Abhängig von Marktakzeptanz und Produktstrategie	Überwachung des gesamten Produktlebenszyklus
Entscheidungsfindung	Kunde gibt Anforderungen vor	Entscheidungen durch internes Produktmanagement	Produktmanager*innen tragen Gesamtverantwortung
Hauptfokus	Kundenorientierte Lösung	Marktgerechtes Produkt	Langfristige Produktstrategie und -planung

Table 1.1: Typen von Software Engineering

Softwareentwicklungsprozesse

Softwareprozess

Satz von Aktivitäten dessen Ziel die Entwicklung oder Weiterentwicklung von Software ist.

Software development process

The process by which user needs are translated into a software product. The process involves translating user needs into software requirements, transforming the software requirements into design, implementing the design in code, testing the code, and sometimes, installing and checking out the software for operational use. (IEEE 610.12)

Naives Vorgehen

Name	Code-and-Fix
Vorgehen	Vorläufiges Programm erstellen; Anforderung, Entwurf, Testen, Wartung überlegen; Programm entsprechend verbessern
Eigenschaften	keine: Spezifikation, Entwurf, nachvollziehbare Systematik
Vorteile	schnell, einfach
Nachteile	nicht planbar, Teamarbeit unmöglich, Programme schlecht strukturiert, keine bestimmten Anforderungen, extrem hoher Korrekturaufwand, schlechte Dokumentation - also schlechte Qualität und teuer

Generelle Schritte

1. Anwendung
 - Nutzung

- Sonderfälle
- Performance
- Technologische oder regulatorische Vorgaben

2. Struktur

- Benutzeroberfläche
- Architektur
- Klassen
- Algorithmik
- Programmiersprache

3. Programmierung

- Modularität
- Nutze Code-Standards und Muster (Erklärbarkeit)
- Standardbibliotheken
- Fehlerbehandlungen

4. Testen

- Korrekte Funktionalität
- Robustheit und Fehlervermeidung
- Performance-Aspekte
- Erweiterbarkeit

5. Aufwandsschätzung

- Aufwände, benötigte Ressourcen und Dauer
- eine Implementierungszeit und Aspekte wie Pausen, Tests, Debugging und reale Faktoren wie Verzögerungen

Beispiel

Name	Wasserfallmodell
Typ	dokumentengetriebenes Modell
Vorgehen	
Eigenschaften	
Vorteile	
Nachteile	

Vorgehens- vs. Prozessmodelle

Name	Vorgehensmodell	Prozessmodell
Umfang	Auszuführende Entwicklungsschritte und deren Reihenfolge	Verfahren zur Anforderungserhebung und Prüfung von Zwischenergebnissen, Projektphasen, Meilensteine und Prüfkriterien, Notationen und Terminologie, Werkzeuge
Fokus	Praktische Umsetzung der Entwicklung	Umfassende Sicht auf den gesamten Softwareentwicklungsprozess
Flexibilität	Oft weniger formal und flexibler in der Anwendung	Strukturierter und formeller Ansatz

Chapter 2

Agiles Software Engineering

2.1 Motive und Prinzipien

Die Motivation ist die Vermeidung unerwünschter Effekte und die Förderung gewünschter Effekte, wie:

- Flexibilität bei sich ändernden Anforderungen
- Höhere Kundenzufriedenheit durch kontinuierliches Feedback
- Verbesserte Teamarbeit und Kommunikation
- Schnellere Lieferzeiten und bessere Qualität: Vermeidung von Zeitverzug
- Vermeidung von Über-Analyse
- Vermeidung von Bürokratisierung des Entwicklungsprozesses

Die Motivation für agile Methoden ist, die Effizienz und Effektivität der Softwareentwicklung zu steigern, indem man:

- Kundenengagement fördert
- Risiken frühzeitig identifiziert und mindert
- Die Anpassungsfähigkeit an sich ändernde Marktbedingungen verbessert

Das Geschieht durch eine Verschlinkung der Prozesse:

- Entwurfs- und Planungsphase minimieren
- Entwicklung fokussiert auf die Softwarefunktionen, die direkten Wert für den späteren Benutzer haben
- Implementierungsphase möglichst frühzeitig starten
- Inkrementelle Entwicklung und Bereitstellung
- Schnell ausführbare Software für Kundenabstimmung
- Frühzeitig und flexibel auf Kundenwünsche eingehen

2.2 Scrum als agiles Prozessmodell

Es existiert eine große Anzahl agiler Methoden und Vorschläge für agile Prozesse! Scrum ist nur eine davon.

Scrum

Scrum ist ein Rahmenwerk für die agile Softwareentwicklung, basierend auf der hyperproduktiven Produktentwicklung. Es wird eingesetzt, wenn Anforderungen sich schnell ändern oder unklar sind und nutzt einen empirischen Ansatz basierend auf der Prozess-Steuerungstheorie.

2.2.1 Überblick

Die Vorteile

- Der **Fortschritt** wird nicht durch instabile Anforderungen aufgehalten
- Das Produkt wird **heruntergebrochen** auf eine Menge von verständlichen Einheiten, zu denen die Stakeholder einen Bezug finden können
- Kunden erhalten Inkremente **pünktlich** geliefert und bekommen Feedback darüber, wie das Produkt arbeitet
- Die **Kommunikation** im Team wird verbessert, weil jeder Einsicht in alles hat
- Es wird **Vertrauen** zwischen Kunden und Entwicklern aufgebaut und eine positive Kultur erzeugt

Die Nachteile

- Hoher **Kommunikationsaufwand**
- **Entwurfsphase fehlt** ⇒ Architekturentwicklung muss explizit berücksichtigt werden
- Eher geeignet für **kleine Teams**
- Herausforderungen bei der **Vertragsgestaltung**

Empirische Prozesssteuerung

- Säulen: Transparenz, Inspektion und Anpassung von Ergebnissen und Aktivitäten
- Kombination von iterativen und inkrementellen Vorgehen
- Grundidee: Entscheidungsspielraum sinkt von Iteration zu Iteration
- Anzahl der messbaren Teilergebnisse steigt stetig
- Unschärfe des Projektziels nimmt im Projektverlauf ab
- Vorteile:
 - Frühe Rückkopplung der Erfahrungen
 - Reduzierung von Risiken in der Umsetzung

Grundprinzipien

- Selbstorganisation
- Pull-Prinzip
- Timebox
- Nutzbare Funktionalität
- Rahmenwerk

Rollen

- **Product Owner (PO)** (Auftraggeber*in)
 -
- **Scrum Master (SM)**
- **Entwicklungsteam**

Artefakte

Product Backlog Items

Product-Backlog-Items (PBIs) beziehen sich auf alle Anforderungen, die im Product Backlog eines Projekts festgehalten werden. Sie können verschiedene Formen annehmen, darunter:

- User Stories
 - Funktionale und nicht funktionale Anforderungen
 - Technische Aufgaben
 - Fehlerbehebungen
-
- **Product Backlog**
 - Anforderungsliste mit Aufgaben
 - Elemente sind sog. **PBIs**
 - sollte immer so priorisiert werden, dass die zuerst zu implementierenden Punkte ganz oben auf der Liste stehen
 - **Produkt-Inkrement**
 - **Product-Backlog-Item**
 - **Vision**
 - **Selected Product Backlog (SPL)**
 - **Sprint Backlog**
 - **Burndown-Chart**
 - **Impediment Backlog**
 - **Sprint Goal**

Ereignisse

- Sprint
 -
- Sprint Planning
- Daily Scrum
 - Kurz
- Sprint Review
- Sprint Retrospective

2.2.2 Scrum Planung

Agile Aufwandsschätzung - Planning Poker

Planning Poker

Voraussetzungen

- Einigen auf [Referenz-Item](#)
- [PBI](#) auswählen

Ablauf

1. Verständnisfragen klären
2. Alle wählen jeweils eine „Spielkarte“ (Fibonacci-Reihe) (verdeckt) aus
3. Alle Karten werden gleichzeitig aufgedeckt
4. Team-Mitglieder mit niedrigster und höchster Wertung erläutern ihre Wahl (moderiert)
5. Wiederholung der Schätzung
6. Sinnvollsten Wert auswählen

Planungshorizonte

Name Typ Inhalt	Release Planning Ungefäher Projektplan Inhaltliche und zeitliche Planung der Softwareauslieferung	Sprint Planning Plan für eine einzelne Iteration Auswahl des Sprint Ziel und des SPL ; Detaillierung der PBIs des SPL mit Entwicklungsaufgaben und Erfassen dieser im Sprint Backlog
Planungshorizont	1-4 Monate	1-4 Wochen
Gemacht von	PO , ggf. Stakeholdern und Entwick- lungsteam	PO , Team und SM

Sprint Planung

$$\text{Priorität} = \text{Wert der Funktionalität} * \text{Risiko} \quad (2.1)$$

- **Analyse-Teil** (was)
 - Vorbereitete Vorstellung der [PBIs](#) / Stories durch [PO](#) gemäß ihrer Priorisierung
 - Entwicklungsteam analysiert jede Story, notiert zusätzliche Anforderungen und Akzeptanzkriterien
 - Realistisches Arbeitspensum des Sprints festlegen ():
 - * Auswahl der [PBIs](#) für Selected Backlog für nächsten Sprint
 - * Team entscheidet auf Basis von Kapazität und [Velocity](#), ob Story noch im Sprint realisierbar ist

$$\text{Gesamtlaufzeit des Backlogs} = \frac{\text{Gesamt-Story-Points}}{\text{Velocity}} \quad (2.2)$$

⇒ [SPL](#) ist festgelegt

- **Design-Teil** (wie)
 - Entwicklungsteam und [SM](#) identifizieren Tasks (konkrete Implementierungsaufgaben) für die Umsetzung der ausgewählten [PBIs](#)

Velocity

[Velocity](#) ist ein Maß für die Arbeitsleistung eines Scrum-Teams und gibt an, wie viele Story Points das Team in einem Sprint erfolgreich abschließt. Es dient als wichtige Kennzahl, um die Effizienz und Produktivität des Teams über mehrere Sprints hinweg zu bewerten.

2.2.3 Durchführung von Sprints

Sprint

Überprüfen des Sprintfortschritts

Burndown-Chart

Die Anzahl der nicht erledigten Arbeiten bzw. des verbleibenden Aufwands, gemessen in Task oder Story Points, visualisiert in einem sog. Restaufwandsdiagramm.

Sprint Nacharbeit

Sprint Review

- Alle Rollen
- Präsentation von Product Increment
- PO entscheidet, ob es akzeptiert wird
- es kann theoretisch alles geändert oder auch verworfen werden

Sprint Retrospektive

- Entwicklungsteam und SM
- Analyse des Sprints
- Positives, Probleme Verbesserungsmöglichkeiten werden diskutiert
- Impediments werden an den SM kommuniziert

2.3 Fragen

Warum werden für die Release-Planung und die Sprintplanung verschiedene Einheiten für die zu schätzenden Aufgaben verwendet?

Was sind Vor- und Nachteile der Schätzung und Messung in Task-Points gegenüber der Verwendung von Stunden?

Was sind Vor- und Nachteile von kurzen (<3 Wochen) und längeren (≥ 4 Wochen) Sprints?

Inwiefern kann der Einsatz von Scrum und agilen Methoden helfen, typische Probleme des Wasserfallmodells zu reduzieren?

Chapter 3

Analyse

3.1 Anforderungen

Funktionale Anforderungen

- Primäre Benutzeraufgaben
- Aktionen, die vom System ausgeführt werden
- Welche Dienste, Eingaben, Ausgaben, Verhalten?
- Was soll nicht passieren?

Nicht funktionale Anforderungen

Chapter 4

Weiteres

Hohe Kohäsion

Hohe Kohäsion bezieht sich auf das Prinzip, dass alle Elemente innerhalb einer Klasse (Methoden und Attribute) eng miteinander verbunden sind und eine gemeinsame Verantwortung oder Aufgabe haben.

Vorteile hoher Kohäsion:

- Bessere Designs: Klassen sind einfacher zu verstehen und zu verwenden.
- Erhöhte Wartungsfähigkeit: Änderungen an einer kohäsiven Klasse betreffen in der Regel nur die Klasse selbst.
- Bessere Wiederverwendbarkeit: Klassen, die klar definierte Aufgaben erfüllen, können leichter in anderen Kontexten verwendet werden.

Was bedeutet lose Kopplung?

Lose Kopplung ist ein Prinzip in der Softwareentwicklung, das darauf abzielt, die Abhängigkeiten zwischen Klassen zu minimieren.

Vorteile loser Kopplung:

- Flexibilität: Klassen können unabhängig voneinander geändert, getestet und gewartet werden.
- Einfachere Wiederverwendbarkeit: Lose gekoppelte Klassen können leichter in anderen Projekten oder Kontexten verwendet werden.
- Erleichterte Wartung: Änderungen in einer Klasse erfordern keine umfangreichen Anpassungen in anderen Klassen.

	MVC	Observer-Pattern
Paradigma	Architektur-Muster	Entwurfsmuster
Zweck	Strukturiert Software in drei Schichten	Ermöglicht Benachrichtigungen bei Änderungen
Kopplung	Lose Kopplung zwischen Model, View, Controller	Sehr lose Kopplung zwischen Subjekt und Beobachtern
Verwendung	Wird für die gesamte Softwarestruktur verwendet	Wird meist zur Implementierung von Benachrichtigungen genutzt
Beziehung zueinander	MVC kann das Observer-Pattern für die Verbindung zwischen Model und View nutzen	Kann in MVC, aber auch unabhängig verwendet werden
Vorteile	Klare Trennung von Verantwortlichkeiten, Erhöhte Wiederverwendbarkeit von Komponenten, Verbesserte Wartbarkeit und Testbarkeit, Ermöglicht parallele Entwicklung von UI und Logik	Ermöglicht lose Kopplung zwischen Subjekt und Beobachtern, Mehrere Views können automatisch aktualisiert werden, Erhöht die Flexibilität bei Änderungen, Einfach zu implementieren und modular erweiterbar
Nachteile	Erhöhter Implementierungsaufwand, komplex, Erfordert oft zusätzliches Design für Datenbindung und Zustandsmanagement	Kann zu Performance-Problemen führen, wenn zu viele Beobachter registriert sind, Zyklische Abhängigkeiten sind möglich, Schwieriger zu debuggen, wenn viele Events gleichzeitig verarbeitet werden

4.0.1 Fragen

Welche Bestandteile hat MVC?

- Model
- View
- Controller

Zeichnen von MVC-Interaktionen

Aktion im View -> View informiert Controller -> Controller verarbeitet Eingabe und aktualisiert Model -> Model ändert Zustand und benachrichtigt View -> View aktualisiert sich

Vorteil(e) des Observer-Patterns?

- Entkopplung
- Flexibilität
- Einfache Erweiterbarkeit

Vorteile von MVC?

- Bessere Wartbarkeit
- Wiederverwendbarkeit
- Bessere Testbarkeit
- Flexibilität

Warum nutzt man Architekturmuster?

- Strukturierung
- Wiederverwendbarkeit
- Wartbarkeit

- Testbarkeit
- Skalierbarkeit

Wie sind Model und View gekoppelt?

Gar nicht, das Model sollte keine direkte Abhängigkeit zur View haben. Aber der View ist der Beobachter des Models.

Warum sollte man keine View-Referenzen im Model speichern?

- Verletzung des MVC-Prinzips
- Schlechte Wartbarkeit
- Schwierige Testbarkeit
- Erweiterbarkeit wird erschwert

Chapter 5

Fragen

5.0.1 Planung bei Scrum

Welche zwei Planungshorizonte gibt es?

- **Langfristige Planung (Release Planning):** Inhaltliche und zeitliche Planung der Softwareauslieferung (welche Features und Funktionalitäten in den kommenden Releases implementiert werden sollen). **1-4 Monate.**
- **Kurzfristige Planung (Sprint Planning):** plant Aktivitäten für eine einzelne Iteration (legt fest, welche Aufgaben und [User Stories](#) im aktuellen [Sprint](#) bearbeitet werden sollen). **1-4 Wochen.**

Wie schätzt und misst man den Gesamtaufwand?

Durch Schätztechniken wie [Planning Poker](#).

- relativen Größen
- braucht Referenz, Einheit und Skala

Gibt es die Möglichkeit, längerfristig zu planen?

Ja, mit **Release Planung** gibt es die Möglichkeit, längerfristig zu planen. Jedoch ist diese Planung flexibler und kann sich während der Iterationen ändern, basierend auf den Ergebnissen und dem Feedback. Langfristige Planungen richten sich oft nach den Releases, die auch die Roadmap des Produkts darstellen. Langfristig planen kann man bspw. durch

- **Identifikation von Releases:** Bestimmung der wichtigsten Releases, die auf der Roadmap des Produkts basieren.
- **Priorisierung von Anforderungen:** Festlegung, welche [User Stories](#) oder Features in den kommenden Releases umgesetzt werden sollen.
- **Schätzung der Implementierungsdauer:** Einschätzung des Aufwands für die Umsetzung der [User Stories](#), um realistische Zeitpläne zu erstellen.
- **Flexibilität in der Planung:** Die Planung sollte anpassbar sein, um auf Veränderungen in den Anforderungen und Feedback aus den [Sprints](#) reagieren zu können.
- **Regelmäßige Überprüfung:** Bei jedem [Sprint Review](#) sollte die langfristige Planung überprüft und angepasst werden, um sicherzustellen, dass sie weiterhin relevant und realistisch ist.

Was umfasst die Planung?

- Identifikation und Priorisierung von Anforderungen ([User Stories](#))
- Schätzung von Aufwand und Risiken
- Zeitplan entlang der Iterationen: Definition von Zielen und Erfolgskriterien für die [Sprints](#)
- Erstellung eines [Sprint-Backlogs](#), das die zu bearbeitenden Aufgaben enthält

Was ist relevant für die Planung?

- Die Erfahrungen aus vorherigen **Sprints**
- Die Teamkapazität und -verfügbarkeit
- Die Prioritäten des **POs**
- Technische Abhängigkeiten und Risiken

Was passiert in einem **Sprint**, in einer Iteration?

- festgelegter Satz an **User Stories** aus dem **Product Backlog** ausgewählt und umgesetzt

Eventuell auch:

- **Daily Scrum**
- Überprüfen des **Sprintfortschritts**

Danach: **Sprint Review** und ein **Sprint Retrospective**

Sollten alle Anforderungen im ersten **Sprint** implementiert werden?

Nein, nicht alle Anforderungen sollten im ersten **Sprint** implementiert werden. Es ist ratsam, **die wichtigsten Anforderungen zuerst** zu bearbeiten, um schnell Feedback zu erhalten und den Fokus auf die wichtigsten Features zu legen.

Was ist die **Velocity** in Scrum?

Die **Velocity** ist ein **Maß** für die **Menge an Arbeit**, die ein Scrum-Team **in einem Sprint** erledigen kann. Sie wird in der Regel in **Story Points (SPs)** oder anderen Maßeinheiten ausgedrückt und hilft Teams, ihre Kapazität für zukünftige **Sprints** zu schätzen.

Wie funktioniert das Release Planning?

Im Release Planning wird festgelegt, welche Features in den nächsten Releases des Produkts implementiert werden. Das Team priorisiert die **User Stories** im **Product Backlog** und plant die Releases basierend auf der geschätzten **Velocity** und den verfügbaren Ressourcen.

Was macht der **PO** ?

Der **PO** ist verantwortlich für das **Management des Product Backlogs**. Dazu gehört das **Priorisieren von Anforderungen**, das Klären von Fragen des Teams und das Sicherstellen, dass die Entwicklungsteams die richtigen Features zur richtigen Zeit umsetzen, um den maximalen Wert für das Unternehmen zu liefern.

Wie belastbar ist eine Schätzung?

Schätzungen in Scrum sind **Annäherungen und nicht genau**. Sie basieren auf dem Wissen und den Erfahrungen des Teams und können durch neue Informationen, Änderungen der Anforderungen oder technische Herausforderungen variieren.

Wie kann man erklären, dass die Ziele realistisch sind?

Die Realisierbarkeit der Ziele kann durch die Analyse der Teamkapazität, vergangene **Sprints** (insbesondere die **Velocity**) und durch Einbeziehung von Stakeholdern und Teammitgliedern in die Schätzungen und Planungen erklärt werden.

Wie belastbar ist ein vorgelegter Zeitplan?

Ein vorgelegter Zeitplan ist belastbar, solange er regelmäßig überprüft und angepasst wird. In Scrum ist es wichtig, flexibel zu bleiben und auf neue Informationen oder Veränderungen im Projektumfeld zu reagieren.

Was ist am Ende des **Sprints** bekannt?

Am Ende des **Sprints** ist bekannt, welche **User Stories** abgeschlossen wurden, ob die **Sprint-Ziele** erreicht wurden und welche Erkenntnisse aus dem **Sprint** gewonnen wurden. Das Team hat ein potenziell lieferbares Produktinkrement erstellt.

Woran lässt sich das ablesen?

Das lässt sich an der Fertigstellung der **User Stories** im **Sprint Backlog**, den Ergebnissen der **Sprint Review** und den Feedback-Runden ablesen. Auch die Retrospektive bietet Einblicke in die Teamdynamik und die Qualität der Arbeit.

5.0.2 Charakterisierung Software-Engineering

Was macht man im Software-Engineering?

Im Software-Engineering werden systematische Methoden und Techniken angewendet, um Softwareprodukte zu entwerfen, zu entwickeln, zu testen und zu warten. Die Hauptaktivitäten umfassen:

- Anforderungsanalyse: Erfassung und Spezifizierung der Anforderungen an die Software.
- Entwurf: Planung der Architektur, Module und Schnittstellen der Software.
- Implementierung: Programmierung und Realisierung der Software basierend auf dem Entwurf.
- Testen: Überprüfung der Software auf Funktionalität, Leistung, Sicherheit und Benutzerfreundlichkeit.
- Wartung: Anpassungen und Verbesserungen nach der Auslieferung, um Fehler zu beheben und neue Anforderungen zu integrieren.

In welche Artefakte werden funktionale Anforderungen überführt?

Funktionale Anforderungen werden in verschiedene Artefakte überführt, darunter:

- **Use Cases**: Beschreiben, wie Benutzer mit dem System interagieren.
- **User Stories**: Kurze Beschreibungen von Anforderungen aus der Perspektive der Benutzer.
- **Lastenheft**: Dokument, das die Anforderungen und Erwartungen des Auftraggebers beschreibt.
- **Pflichtenheft**: Detaillierte Beschreibung, wie die Anforderungen erfüllt werden sollen, einschließlich technischer Spezifikationen.
- Prototypen: Visuelle Darstellungen der Software, um Anforderungen zu testen und Feedback zu sammeln.

Wo findet das Anforderungsmanagement statt?

Das Anforderungsmanagement findet in der Regel in der Phase der Anforderungsanalyse statt, die Teil des gesamten Softwareentwicklungsprozesses ist. Dies geschieht in enger Zusammenarbeit zwischen Stakeholdern, Kunden und dem Entwicklungsteam. Es wird oft in einem agilen Umfeld kontinuierlich während des gesamten Lebenszyklus des Projekts durchgeführt, um sicherzustellen, dass die Anforderungen aktuell und relevant bleiben.

Wie ist eine **User Story Card (USC)** aufgebaut?

Eine **USC** ist typischerweise einfach strukturiert und enthält folgende Elemente:

- Titel: Eine kurze Beschreibung der **User Story**.
- Beschreibung:
 - Wer: Der Benutzer oder die Benutzergruppe (Akteur).
 - Was: Die Funktionalität oder das gewünschte Ziel.
 - Warum: Der Nutzen oder die Motivation hinter der Anforderung.
- Akzeptanzkriterien: Bedingungen, die erfüllt sein müssen, damit die **User Story** als abgeschlossen gilt.
- Schätzung: Eine Schätzung des Aufwands (z.B. in **SPs**).

Welche Methoden zur Aufwandschätzung gibt es?

Es gibt verschiedene Methoden zur Aufwandschätzung, darunter:

- **Planning Poker:** Eine Schätztechnik, bei der Teammitglieder Karten mit Schätzungen verwenden.
- **T-Shirt-Größen:** Eine einfache Methode, um Anforderungen in Größen (XS, S, M, L, XL) zu kategorisieren.
- **Delphi-Methode:** Eine strukturierte Kommunikationsmethode zur Schätzung durch Experten.
- **Analogie-Schätzung:** Vergleich mit ähnlichen, bereits durchgeführten Projekten oder Aufgaben.
- **Punkteschätzung:** Verwendung von **SPs** zur Schätzung der Komplexität.

Erfassung von Anforderungen

Die Anforderungen werden typischerweise in folgende Kategorien unterteilt:

- **Funktionale Anforderungen:** Beschreiben, was das System tun soll.
- **Nicht-funktionale Anforderungen:** Beschreiben die Qualitätsmerkmale des Systems, wie z.B. Leistung, Sicherheit, Usability und Wartbarkeit.
- **Technische Anforderungen:** Beziehen sich auf die technischen Aspekte, die für die Implementierung notwendig sind.
- **Geschäftliche Anforderungen:** Umfassen die Ziele und Bedürfnisse des Unternehmens.

5.0.3 Herausforderungen bei der Anforderungsanalyse

Kann man Anforderungen systematisch erfassen?

Ja, Anforderungen können systematisch erfasst werden. Methoden wie Interviews, Workshops, Umfragen, Beobachtungen und die Analyse bestehender Dokumente helfen dabei, die Bedürfnisse und Erwartungen der Stakeholder zu verstehen. Techniken wie **Use Cases**, **User Stories** und **Personas** werden oft verwendet, um Anforderungen klar zu definieren und zu dokumentieren.

Was ist das Format eines Anwendungsfalls?

Ein Anwendungsfall (Use Case) beschreibt eine Interaktion zwischen einem Benutzer (Akteur) und einem System, um ein bestimmtes Ziel zu erreichen. Das Format eines Anwendungsfalls umfasst typischerweise:

- **Titel:** Name des Anwendungsfalls
- **Akteure:** Wer interagiert mit dem System?
- **Ziel:** Was soll erreicht werden?
- **Vorbedingungen:** Bedingungen, die erfüllt sein müssen, bevor der Anwendungsfall gestartet werden kann.
- **Ablauf:** Eine Schritt-für-Schritt-Beschreibung der Interaktion.
- **Nachbedingungen:** Zustand des Systems nach der Durchführung des Anwendungsfalls.
- **Alternativabläufe:** Abweichungen oder Fehlerbehandlungen während des Ablaufs.

Sind Anwendungsfälle nur für den Entwurf geeignet?

Nein, Anwendungsfälle sind nicht nur für den Entwurf geeignet. Sie dienen auch der Anforderungsanalyse, da sie helfen, die Anforderungen an das System aus der Perspektive der Benutzer zu verstehen. Anwendungsfälle können in verschiedenen Phasen des Softwareentwicklungsprozesses verwendet werden, einschließlich der Analyse, des Designs und der Validierung.

Wie erfasst man Anforderungen?

Anforderungen können durch verschiedene Methoden erfasst werden, darunter:

- **Interviews mit Stakeholdern:** Direkte Gespräche, um die Bedürfnisse zu verstehen.
- **Workshops und Brainstorming-Sitzungen:** Gemeinsame Diskussionen zur Erfassung von Ideen und Anforderungen.
- **Beobachtungen:** Analyse der aktuellen Arbeitsabläufe und Systeme der Benutzer.
- **Dokumentenanalysen:** Überprüfung vorhandener Dokumentationen und Systembeschreibungen.
- **Prototyping:** Erstellung von Modellen oder Mockups, um Feedback zu den Anforderungen zu erhalten.

Wenn der Kunde nicht alles beschreiben/erklären kann - wie geht man vor?

Wenn der Kunde nicht alle Anforderungen klar beschreiben kann, kann man folgende Ansätze verfolgen:

- **Interaktive Workshops:** Durch gezielte Fragen und Diskussionen können Anforderungen schrittweise erarbeitet werden.
- **Prototyping:** Visuelle oder funktionale Prototypen helfen, die Vorstellungen des Kunden zu konkretisieren.
- **Iterative Rückmeldungen:** Regelmäßige Überprüfungen und Feedback-Runden während des Entwicklungsprozesses, um Missverständnisse zu klären und Anforderungen zu verfeinern.
- **Verwendung von Personas:** Erstellung von Personas, um die Benutzerbedürfnisse besser zu verstehen und zu kommunizieren.

Was soll die Software am Ende alles können?

Die Software sollte am Ende die definierten funktionalen und nicht-funktionalen Anforderungen erfüllen. Dazu gehören:

- **Erfüllung der Benutzerbedürfnisse und -erwartungen.**
- **Bereitstellung der notwendigen Funktionen und Features.**
- **Sicherstellung von Qualität, Leistung, Sicherheit, Usability und Wartbarkeit.**

Lasten- und Pflichtenheft

- **Lastenheft:** Dokument, das die Anforderungen und Erwartungen des Auftraggebers an das System beschreibt. Es enthält die Ziele, Funktionalitäten und Rahmenbedingungen.
- **Pflichtenheft:** Detaillierte Beschreibung, wie die Anforderungen aus dem Lastenheft umgesetzt werden sollen. Es enthält technische Spezifikationen, Systemarchitektur und Implementierungsdetails.

Welche Aktivitäten gibt es, um die Analyse durchzuführen?

Aktivitäten zur Durchführung der Anforderungsanalyse können Folgendes umfassen:

- **Interviews und Umfragen:** Um die Meinungen und Bedürfnisse der Stakeholder zu erfassen.
- **Workshops:** Gemeinsame Diskussionen zur Ideenfindung und Anforderungsdefinition.
- **Prototyping:** Entwicklung von Prototypen, um Anforderungen zu testen und Feedback zu sammeln.
- **Erstellung von Personas:** Nutzung von Personas, um die Benutzerbedürfnisse zu verstehen.
- **Nutzung von User Stories:** Erfassung der Anforderungen in Form von User Stories.

Unterschied zwischen **User Story** und **Szenario**?

- **User Story:** Eine kurze, einfache Beschreibung einer Funktionalität aus der Sicht des Endbenutzers, oft im Format: "Als [Benutzertyp] möchte ich [Ziel], damit [Nutzen]." **User Stories** sind in der Regel weniger detailliert und bieten Raum für Diskussion und Anpassungen.
- **Szenario:** Ein **Szenario** ist eine detailliertere Beschreibung einer spezifischen Situation, in der ein Benutzer mit dem System interagiert. Es beschreibt den Kontext und die Schritte einer Interaktion und kann mehrere **User Stories** umfassen.

5.0.4 Struktur eines Anwendungsfalls

Wie ist das Dokument des **Use Case** strukturiert?

- **Titel:** Eine klare und prägnante Bezeichnung des Anwendungsfalls.
- **Ziel:** Eine kurze Beschreibung des Ziels des Anwendungsfalls und was erreicht werden soll.
- **Akteure:** Die Benutzer oder Systeme, die mit dem Anwendungsfall interagieren, darunter:
 - Primäre Akteure: Die Hauptbenutzer, die die Funktionalität nutzen.
 - Sekundäre Akteure: Andere Systeme oder Personen, die mit dem Anwendungsfall interagieren.
- **Vorbedingungen:** Bedingungen, die erfüllt sein müssen, bevor der Anwendungsfall gestartet werden kann.
- **Ablauf:** Eine Schritt-für-Schritt-Beschreibung der Interaktion zwischen dem Akteur und dem System, einschließlich Hauptablauf und Alternativabläufe.
- **Nachbedingungs:** Der Zustand des Systems nach der Durchführung des Anwendungsfalls.
- **Akzeptanzkriterien:** Die spezifischen Bedingungen, die erfüllt sein müssen, damit der Anwendungsfall als erfolgreich abgeschlossen gilt.
- **Besondere Anforderungen:** Zusätzliche Informationen oder Anforderungen, die für den Anwendungsfall relevant sind.
- **Ereignisse:** Eine Liste von Ereignissen oder Triggern, die den Anwendungsfall auslösen können.

5.0.5 Agile Softwareentwicklung

Wie geht man in einem agilen Prozess vor?

In einem agilen Prozess folgt man einem iterativen und inkrementellen Ansatz, der es Teams ermöglicht, flexibel auf Änderungen zu reagieren und kontinuierlich Feedback zu integrieren. Die Schritte umfassen typischerweise:

- Planung (**Sprint Planning**)
- Durchführung (**Sprint Execution**)
- Überprüfung (**Sprint Review**)
- Retrospektive (**Sprint Retrospective**)

Was sind Kennzeichen von Scrum?

Die Kennzeichen von Scrum umfassen:

- Rollen: **PO** , **SM**, Entwicklungsteam.
- Artefakte: **Product Backlog**, **Sprint Backlog**, Increment.
- Ereignisse: **Sprint**, **Sprint Planning**, **Daily Scrum**, **Sprint Review**, **Sprint Retrospective**.
- Iterative Entwicklung: Regelmäßige **Sprints** zur Lieferung von funktionsfähiger Software.

Was ist das Prozessmodell?

Das Prozessmodell in Scrum beschreibt den iterativen und inkrementellen Ansatz zur Softwareentwicklung. Es beinhaltet:

- Kurze Entwicklungszyklen (**Sprints**)
- Regelmäßige Überprüfung und Anpassung
- Fokussierung auf die Lieferung von Wert für den Kunden.

Welche Motive gibt es für agile Methoden?

Motive für agile Methoden sind:

- Flexibilität bei sich ändernden Anforderungen
- Höhere Kundenzufriedenheit durch kontinuierliches Feedback
- Verbesserte Teamarbeit und Kommunikation
- Schnellere Lieferzeiten und bessere Qualität: Vermeidung von Zeitverzug
- Vermeidung von Über-Analyse
- Vermeidung von Bürokratisierung des Entwicklungsprozesses

Welche Motivation gibt es für agile Methoden?

Die Motivation für agile Methoden ist, die Effizienz und Effektivität der Softwareentwicklung zu steigern, indem man:

- Kundenengagement fördert
- Risiken frühzeitig identifiziert und mindert
- Die Anpassungsfähigkeit an sich ändernde Marktbedingungen verbessert.

Wie sieht grob eine zeitliche Planung aus?

Die zeitliche Planung in Scrum erfolgt in **Sprints**, die in der Regel 1 bis 4 Wochen dauern. Grob sieht die Planung folgendermaßen aus:

- **Sprint Planning**: Festlegung der Ziele und des Backlogs für den **Sprint**.
- **Daily Scrum**: Kurze tägliche Meetings zur Abstimmung.
- **Sprint Review**: Präsentation der Ergebnisse am Ende des **Sprints**.
- **Sprint Retrospective**: Reflexion über den Prozess zur kontinuierlichen Verbesserung.

Warum wurden die Prinzipien der agilen Software-Entwicklung entwickelt?

Die Prinzipien der agilen Software-Entwicklung wurden entwickelt, um:

- Die Reaktionsfähigkeit auf Veränderungen zu verbessern.
- Die Zusammenarbeit zwischen Stakeholdern zu fördern.
- Die Qualität der Software durch kontinuierliches Feedback zu erhöhen.

Wie kann man agile Prinzipien umsetzen?

Agile Prinzipien können umgesetzt werden durch:

- Regelmäßige Kommunikation und Zusammenarbeit im Team.
- Iteratives Arbeiten mit kurzen Feedbackzyklen.
- Priorisierung von Anforderungen basierend auf Kundenwert.
- Nutzung von Werkzeugen zur Unterstützung der agilen Praktiken.

Wie geht man agil vor?

Agil vorzugehen bedeutet, flexibel und anpassungsfähig zu sein, während man:

- In kurzen Zyklen arbeitet.
- Regelmäßige Anpassungen an den Anforderungen vornimmt.
- Kontinuierliches Feedback von den Stakeholdern einholt.

Was sind mögliche Vorgehensweisen?

- Scrum
- Kanban
- Extreme Programming (XP)
- Feature Driven Development (FDD)

Was meint das iterativ-inkrementelle Vorgehen?

- Iteration: Wiederholung von Entwicklungszyklen, um schrittweise Fortschritte zu erzielen.
- Inkrementell: Lieferung von Software in kleinen, funktionsfähigen Teilen (Inkrementen).

Hat der Auftraggeber Planbarkeit bei agilen Prozessen?

Ja, der Auftraggeber hat Planbarkeit, jedoch ist diese flexibler und anpassungsfähiger.

Hat man mit agilen Methoden zeitliche Konsistenz?

Mit agilen Methoden kann zeitliche Konsistenz erreicht werden, indem:

- Die **Velocity** des Teams gemessen wird.
- Regelmäßige Schätzungen und Anpassungen stattfinden.

Welche Rolle spielt die Performance des Teams?

Die Performance des Teams ist entscheidend für den Erfolg agiler Projekte, da sie direkten Einfluss auf die Geschwindigkeit und Qualität der Ergebnisse hat.

Wie misst man die Performance des Teams?

Die Performance des Teams kann gemessen werden durch:

- **Velocity**: Anzahl der **SPs**, die in einem **Sprint** abgeschlossen werden.
- Burndown Charts: Visualisierung des verbleibenden Aufwands im **Sprint**.
- Teamzufriedenheit: Regelmäßige Feedback-Runden und Retrospektiven.

Wie wird **Velocity** im **Sprint Planning** genutzt?

- Um die Anzahl der **User Stories** zu bestimmen, die in den nächsten **Sprint** aufgenommen werden können.
- Um eine realistische Schätzung des Arbeitsaufwands zu geben.

5.0.6 Kopplung von Klassen

Wie können Klassen miteinander gekoppelt sein?

Klassen können auf verschiedene Weise gekoppelt sein, darunter:

- **Direkte Kopplung:** Eine Klasse verwendet direkt die Methoden oder Attribute einer anderen Klasse.
- **Indirekte Kopplung:** Eine Klasse nutzt eine dritte Klasse, um mit einer anderen Klasse zu interagieren.
- **Datenkopplung:** Klassen kommunizieren über gemeinsame Datenstrukturen.
- **Kontrollkopplung:** Eine Klasse steuert das Verhalten einer anderen Klasse durch Übergabe von Steuerinformationen.

Wie können Klassen sich gegenseitig benutzen?

Klassen können sich gegenseitig benutzen, indem sie:

- Methoden der anderen Klasse aufrufen.
- Objekte der anderen Klasse erstellen und deren Methoden verwenden.
- Daten austauschen, indem sie gemeinsame Datenstrukturen oder **Interfaces** nutzen.

Was bedeutet hohe Kopplung?

High Coupling bedeutet, dass Klassen stark voneinander abhängig sind. Änderungen in einer Klasse können erhebliche Auswirkungen auf andere Klassen haben, was die Wartbarkeit und Flexibilität des Systems verringert.

Warum sollte man sich darüber Gedanken machen?

Man sollte sich Gedanken über die **Kopplung** machen, weil:

- **High Coupling** die Wartbarkeit und Testbarkeit des Codes erschwert.
- Änderungen in einem Teil des Systems unvorhergesehene Fehler in anderen Teilen verursachen können.
- Eine lose **Kopplung** die Flexibilität und Wiederverwendbarkeit eines Systems erhöht.

Was sind die GRASP-Pattern?

GRASP (General Responsibility Assignment Software Patterns) sind Muster zur Zuweisung von Verantwortlichkeiten in der Softwareentwicklung. Einige der wichtigsten **GRASP-Pattern** sind:

- **Information Expert:** Verantwortlichkeiten sollten den Klassen zugewiesen werden, die die benötigten Informationen besitzen.
- **Creator:** Bestimmt, welche Klasse für die Erstellung eines Objekts verantwortlich sein sollte.
- **Controller:** Eine Klasse, die als Koordinator für Benutzerinteraktionen fungiert.

Was sind die GRASP-Prinzipien

- **Information Expert:** Verantwortlichkeiten sollten den Klassen zugewiesen werden, die die benötigten Informationen besitzen.
- **Creator:** Bestimmt, welche Klasse für die Erstellung eines Objekts verantwortlich sein sollte.
- **Controller:** Eine Klasse, die als Koordinator für Benutzerinteraktionen fungiert.
- **Low Coupling:** Minimiert die Abhängigkeiten zwischen Klassen.
- **High Cohesion:** Fördert die Zusammengehörigkeit von Aufgaben innerhalb einer Klasse.

Stehen Kohäsion und Kopplung im Widerspruch?

Ja, **Kohäsion** und **Kopplung** stehen oft im Widerspruch zueinander. **High Cohesion** innerhalb einer Klasse bedeutet, dass die Verantwortlichkeiten gut zusammenpassen, während **Low Coupling** zwischen Klassen bedeutet, dass die Klassen unabhängig voneinander arbeiten.

Wie kann man feststellen, ob eine Klasse kohäsiv ist?

Eine Klasse ist kohäsiv, wenn:

- Alle Methoden und Attribute der Klasse eng miteinander verbunden sind.
- Die Klasse eine klar definierte Rolle im System hat.
- Die Funktionalität der Klasse aus Sicht des Benutzers sinnvoll erscheint.

Kann man Metriken wie [Low Coupling](#) oder [High Cohesion](#) berechnen?

Ja, es gibt Metriken zur quantitativen Bewertung von [Kohäsion](#) und [Kopplung](#):

- Kohäsionsmetriken: LCOM (Lack of Cohesion of Methods) bewertet, wie eng die Methoden einer Klasse zusammenarbeiten.
- Kopplungsmetriken: CBO (Coupling Between Objects) bewertet, wie viele andere Klassen mit einer bestimmten Klasse interagieren.

Nennen Sie Beispiele aus dem Begleitprojekt?**Wie kann eine [Aggregation](#) aussehen?**

[Aggregation](#) ist eine Form der Beziehung zwischen Klassen, bei der eine Klasse eine andere als Teil von sich enthält, jedoch unabhängig von ihr ist. Beispiel:

- Eine Library-Klasse kann eine Sammlung von Book-Objekten aggregieren.

Wofür werden [Interfaces](#) genutzt?

[Interfaces](#) werden verwendet, um:

- Die Implementierung einer Klasse von ihrer Verwendung zu trennen.
- Eine gemeinsame Schnittstelle zu definieren, die von verschiedenen Klassen implementiert werden kann.
- [Low Coupling](#) zwischen Klassen zu fördern.

Warum ist [Kopplung](#) ein Problem?

[Kopplung](#) ist ein Problem, weil:

- [High Coupling](#) die Wartbarkeit des Codes verringert.
- Änderungen in einer Klasse unvorhergesehene Auswirkungen auf andere Klassen haben können.
- Die Wiederverwendbarkeit von Klassen erschwert wird.

5.0.7 GRASP**Warum gibt es die Richtlinien?**

Die GRASP-Richtlinien wurden entwickelt, um Softwarearchitekten und Entwicklern zu helfen, Verantwortlichkeiten effektiv zuzuweisen.

Was sind Verantwortlichkeiten?

Verantwortlichkeiten sind die Aufgaben oder Funktionen, die einer Klasse oder einem Objekt in einem Software-System zugewiesen werden.

Was ist der Kernaspekt von GRASP?

Der Kernaspekt von GRASP ist die Zuweisung von Verantwortlichkeiten an Klassen und Objekte in einem objektorientierten Design.

Woher kommen die Verantwortlichkeiten für den objektorientierten Entwurf?

Die Verantwortlichkeiten stammen aus:

- Anforderungen
- Use Cases
- Domänenmodell

Wie helfen die Patterns beim Entwurf?

Die GRASP-Patterns helfen beim Entwurf, indem sie:

- Verantwortlichkeiten klären
- Kohäsion und Kopplung verbessern
- Best Practices bereitstellen

Wann kommen GRASP-Patterns zum Einsatz?

GRASP-Patterns kommen zum Einsatz, wenn:

- Ein neues System entworfen wird
- Bestehende Systeme analysiert oder refaktoriert werden

Woher weiß man, was eine Klasse können muss?

Man weiß, was eine Klasse können muss, indem man:

- Use Cases analysiert
- CRC-Kartes verwendet

Patterns: Low Coupling, High Cohesion, Information Expert, Creator Pattern

- **Low Coupling**: Minimiert die Abhängigkeiten zwischen Klassen.
- **High Cohesion**: Fördert die Zusammengehörigkeit von Aufgaben innerhalb einer Klasse.
- **Information Expert**: Verantwortlichkeiten werden den Klassen zugewiesen, die die benötigten Informationen besitzen.
- **Creator Pattern**: Bestimmt, welche Klasse für die Erstellung eines Objekts verantwortlich sein sollte.
- **Welche Verantwortlichkeiten werden damit abgegeben?**
 - **Information Expert**: Überträgt die Verantwortung für die Verarbeitung von Informationen an die Klasse, die die Daten besitzt.
 - **Creator**: Delegiert die Verantwortung für die Erstellung von Objekten an die Klasse, die die Daten hat.
 - **Controller**: Überträgt die Verantwortung für die Benutzerinteraktion an eine spezielle Klasse.
 - **Low Coupling**: Minimiert die Verantwortung für die Interaktion zwischen Klassen.
 - **High Cohesion**: Gibt spezifische Aufgaben an Klassen weiter.
 - **Polymorphism** : Überträgt die Verantwortung für die Behandlung von Objekten an eine gemeinsame Schnittstelle.
 - **Pure Fabrication** : Kapselt spezifische Verantwortlichkeiten in einer neuen Klasse.
 - **Indirection**: Delegiert Interaktionen an eine Vermittlerklasse.
 - **Protected Variations**: Schützt vor Variationen durch den Einsatz von Abstraktionen.
- **Erläutern, wie die Pattern funktionieren.**
 - **Information Expert**: Weist Verantwortlichkeiten der Klasse zu, die die relevanten Daten hat.

- **Creator**: Bestimmt, dass die Klasse, die ein Objekt benötigt, auch für dessen Erstellung verantwortlich ist.
- **Controller**: Koordiniert Benutzerinteraktionen.
- **Low Coupling**: Verringert Abhängigkeiten zwischen Klassen.
- **High Cohesion**: Fördert die Zusammengehörigkeit von Funktionen innerhalb einer Klasse.
- **Polymorphism** : Ermöglicht eine einheitliche Behandlung verschiedener Klassen durch gemeinsame Schnittstellen.
- **Pure Fabrication** : Schafft eine Klasse zur Kapselung spezifischer Verantwortlichkeiten.
- **Indirection**: Verwendet eine Vermittlerklasse zur Reduzierung der Kopplung.
- **Protected Variations**: Schützt das System durch den Einsatz von Schnittstellen oder Abstraktionen.

Welche Klassen haben die höchste Priorität?

Die Klassen mit der höchsten Priorität sind:

- Klassen, die grundlegende Geschäftslogik oder kritische Funktionen implementieren.
- Klassen, die häufig in **Use Cases** verwendet werden.

Welche Klasse instanziiert Objekte?

In der Regel instanziiert die Klasse, die für die Erstellung und Verwaltung des Objekts verantwortlich ist, die Objekte.

Was ist **Pure Fabrication** ?

Pure Fabrication ist ein **GRASP-Pattern**, das vorschlägt, eine Klasse zu erstellen, die nicht direkt aus den Anforderungen abgeleitet ist, um die **Kopplung** zu reduzieren.

- Beispiel aus dem Begleitprojekt?

Beispiel aus dem Begleitprojekt?

Was hat die Anforderungsanalyse mit **GRASP** zu tun?

Die Anforderungsanalyse hat mit **GRASP** zu tun, weil **GRASP-Richtlinien** helfen, die Verantwortlichkeiten basierend auf den Anforderungen zu definieren.

Wie helfen **GRASP-Pattern**, die Verantwortlichkeiten umzusetzen?

GRASP-Patterns helfen, die Verantwortlichkeiten umzusetzen, indem sie klare Richtlinien für die Zuweisung von Verantwortlichkeiten bieten.

Wie können Klassen in Java gekoppelt sein?

Klassen in Java können auf verschiedene Weise gekoppelt sein:

- **Direkte Kopplung**
- **Indirekte Kopplung**
- **Datenkopplung**
- **Kontrollkopplung**

5.0.8 **CRC-Kartes**

Wie ist eine **CRC-Karte** aufgebaut?

Eine **CRC-Karte** ist typischerweise in folgende Elemente gegliedert:

- **Klasse**: Der Name der Klasse, die definiert wird.
- **Verantwortlichkeiten**: Eine Liste von Aufgaben oder Funktionen, für die die Klasse verantwortlich ist.
- **Zusammenarbeit**: Eine Liste von anderen Klassen oder Objekten, mit denen die Klasse interagiert.

Wie finde ich die Verantwortlichkeiten heraus?

Verantwortlichkeiten können durch folgende Methoden ermittelt werden:

- Analyse von **Use Cases**
- Interviews mit Stakeholdern
- Brainstorming-Sitzungen
- Dokumentenanalysen

Was versteht man unter einer Verantwortlichkeit und in welchem Dokument steht sie?

- **Verantwortlichkeit:** Eine Aufgabe oder Funktion, die einer Klasse zugewiesen ist.
- **Dokumente:** Verantwortlichkeiten sind typischerweise in folgenden Dokumenten festgehalten:
 - **Use Case-Dokumente**
 - **Lastenhefte**
 - **Pflichtenhefte**

5.0.9 High Cohesion

Was bedeutet Low Coupling?

Low Coupling ist ein Prinzip in der Softwareentwicklung, das darauf abzielt, die Abhängigkeiten zwischen Klassen zu minimieren.

5.0.10 BBTs

Kann man in BBTs den Code sehen?

Nein, in **BBTs** sieht man den Code nicht. Die Tests konzentrieren sich ausschließlich auf die Eingaben und Ausgaben eines Systems.

Gibt es Äquivalenzklassen?

Ja, in **BBTs** gibt es Äquivalenzklassen. Diese sind:

- **Gültige Äquivalenzklassen:** Eingabewerte, die gültig sind und vom System akzeptiert werden sollten.
- **Ungültige Äquivalenzklassen:** Eingabewerte, die ungültig sind und vom System abgelehnt werden sollten.

5.0.11 GBTs

Unterschied zum BBT

GBTs (oder White-Box-Tests) unterscheiden sich von **BBTs** dadurch, dass der Tester Zugriff auf den Quellcode hat und die interne Logik der Anwendung versteht. Während **BBTs** sich auf die Eingaben und Ausgaben konzentrieren, analysieren **GBTs** die Struktur und den Ablauf des Codes.

Wie sollte ein GBT konstruiert werden?

Ein **GBT** sollte konstruiert werden, indem:

- Der Quellcode analysiert wird, um alle möglichen Ausführungspfade zu identifizieren.
- Testfälle erstellt werden, die alle signifikanten **Pfade**, Bedingungen und Schleifen im Code abdecken.

Welche Tabellen oder Diagramme kann man verwenden?

Folgende Tabellen oder Diagramme können verwendet werden:

- **Kontrollflussdiagramms**
- **Entscheidungsdiagramme**
- Testfall-Tabellen

Was sind Überdeckungsmaße und welche gibt es?

Überdeckungsmaße sind Metriken, die den Grad der Codeabdeckung während der Tests angeben. Es gibt verschiedene Arten von Überdeckungsmaßen:

- **Anweisungsüberdeckung:** Misst, wie viele Anweisungen im Code ausgeführt wurden.
- **Zweigüberdeckung:** Misst, wie viele **Entscheidungspunkte** (z.B. if-Anweisungen) im Code durchlaufen wurden.
- **Pfadüberdeckung:** Misst, wie viele verschiedene **Pfads** im **Kontrollflussdiagramm** durchlaufen wurden.

(Beispiel von den Übungszetteln)

Welche **Detektors** kann es geben?

Detektors können Folgendes umfassen:

- Syntax-Checker
- Code-Analyse-Tools
- Testfall-Generatoren

Wie wird die for-Schleife behandelt?

In einem **GBT** wird die for-Schleife behandelt, indem man sicherstellt, dass alle möglichen Iterationen der Schleife getestet werden, einschließlich der Fälle, in denen die Schleife nicht ausgeführt wird (z.B. wenn die Bedingung von Anfang an falsch ist).

Sind das unterschiedliche **Pfads** oder **Zweigs**?

Unterschiedliche Zweige: Schleife wird ausgeführt oder nicht.

Unterschiedliche Pfade: die verschiedenen Iterationen, die überprüft werden

Was ist ein **Pfad**, was ist ein **Zweig**?

- **Pfad:** Eine vollständige Sequenz von Anweisungen, die während der Ausführung eines Programms durchlaufen wird.
- **Zweig:** Ein Entscheidungs- oder Kontrollpunkt im Code, an dem der Fluss je nach Bedingung in verschiedene Richtungen verlaufen kann.

Wann nutzt man **GBTs** ?

GBTs werden verwendet, wenn:

- Die interne Logik und Struktur des Codes bekannt ist.
- Eine detaillierte Überprüfung der Implementierung erforderlich ist.
- Man sicherstellen möchte, dass alle Codepfade getestet werden.

Was sind die Vorteile eines **GBTs** gegenüber eines **BBTs**?

Die Vorteile sind:

- Bessere Erkennung von Logikfehlern.
- Höhere Testabdeckung durch gezielte Tests aller Codepfade.
- Möglichkeit, die Testeffizienz zu erhöhen, indem redundante Tests vermieden werden.

Wie kann man Code-Abschnitte systematisch testen?

Code-Abschnitte können systematisch getestet werden, indem:

- Testfälle für alle möglichen **Pfads** erstellt werden.
- Überdeckungsmaße verwendet werden, um sicherzustellen, dass alle Teile des Codes getestet werden.

(Zeichnen eines Kontrollflussgraphen)

Benennung von Elementen

Unterschied zwischen **Pfad-** und **Zweigüberdeckung**

- **Pfadüberdeckung:** Misst, ob alle möglichen **Pfads** durch den Code getestet wurden.
- **Zweigüberdeckung:** Misst, ob alle Entscheidungszweige (z.B. if-Anweisungen) durchlaufen wurden.

Unterschied zwischen **Pfadsn** und **Äquivalenzklassen**

- **Pfads:** Repräsentieren die Ausführungssequenzen im Code.
- **Äquivalenzklassen:** Gruppen von Eingaben, die das gleiche Verhalten des Systems erwarten lassen.

Termbäume

Termbäume sind eine grafische Darstellung von Ausdrücken, die in vielen Programmiersprachen verwendet werden. Sie helfen bei der Analyse von Ausdrücken in Bezug auf die Ausführungsreihenfolge und Priorität von Operationen.

Was sind **Äquivalenzklassen**?

Äquivalenzklassen sind Gruppen von Eingabewerten, die dasselbe Verhalten des Systems erwarten lassen, um die Anzahl der notwendigen Testfälle zu reduzieren.

Wie wird starke **Äquivalenz** sichtbar?

Starke **Äquivalenz** zwischen Testfällen wird sichtbar, wenn alle relevanten Eingabewerte und Zustände eines Systems getestet werden, um sicherzustellen, dass es in jeder Situation korrekt funktioniert.

Ist immer eine 100%-ige **Pfadüberdeckung** möglich?

Eine 100%-ige **Pfadüberdeckung** ist in der Praxis oft schwierig zu erreichen, da es möglicherweise zu viele Kombinationen von Bedingungen gibt, insbesondere in komplexen Systemen.

Was ist die Grundlage für einen **GBT** ?

Die Grundlage für einen **GBT** ist das Verständnis des Quellcodes, der Logik und der Struktur des Systems, um gezielte Tests zu erstellen.

Wie kommt man vom Graphen zu systematischen Tests?

Man kommt vom Kontrollflussgraphen zu systematischen Tests, indem man die **Pfads** und **Zweigs** im Graphen analysiert und Testfälle entwickelt, die diese abdecken.

Wo wendet man **GBTs** an?

GBTs werden in der Regel in der Unit-Testing-Phase und bei der Integrationstestphase angewendet.

Wo lohnt sich der Einsatz?

Der Einsatz von **GBTs** lohnt sich besonders in sicherheitskritischen Anwendungen, wo die interne Logik des Codes entscheidend für die Funktionalität ist.

Für welchen Zweck eignet sich ein **GBT** ?

GBTs eignen sich für den Zweck, die interne Logik von Software zu verifizieren, um sicherzustellen, dass alle **Pfads** und Bedingungen korrekt implementiert sind.

Wie erreiche ich alle Anweisungen?

Um alle Anweisungen zu erreichen, sollten Tests so gestaltet werden, dass sie jeden möglichen **Pfad** im Code abdecken, einschließlich aller Bedingungen und Schleifen.

5.0.12 Domänenmodell

Was ist ein **Domänenmodell** und warum ist es hilfreich?

Ein **Domänenmodell** ist eine abstrahierte Darstellung der relevanten Konzepte, Beziehungen und Regeln innerhalb eines bestimmten Anwendungsbereichs. Es hilft, die Anforderungen besser zu verstehen und zu kommunizieren, weil:

- es eine gemeinsame Sprache für alle Stakeholder schafft.
- es hilft, die Komplexität der Domäne zu reduzieren.
- es als Grundlage für die Softwarearchitektur und das Design dient.
- es die Identifizierung von Entitäten, deren Attribute und deren Beziehungen ermöglicht.

Was ist eine **semantische Lücke**?

Eine **semantische Lücke** tritt auf, wenn es einen Unterschied zwischen dem, was das System tatsächlich tut, und dem, was die Stakeholder erwarten oder benötigen.

Wo tritt sie auf?

Semantische Lücken können an verschiedenen Stellen im Softwareentwicklungsprozess auftreten, darunter:

- Während der Anforderungsanalyse.
- Bei der Erstellung des **Domänenmodells**.
- In der Implementierungsphase.

Was enthält ein **Domänenmodell**?

Ein **Domänenmodell** enthält typischerweise:

- Entitäten: Hauptobjekte oder Konzepte in der Domäne.
- Attribute: Eigenschaften oder Merkmale der Entitäten.
- Beziehungen: Verbindungen zwischen den Entitäten.
- Regeln: Geschäftsregeln und Einschränkungen.

Wie kommt man zum Software-Design?

Der Übergang vom **Domänenmodell** zum Software-Design erfolgt typischerweise in mehreren Schritten:

- Analyse des **Domänenmodells**.
- Identifikation von Klassen und Objekten.
- Definition von Schnittstellen und Interaktionen.
- Erstellung von Prototypen.
- Iterative Verfeinerung.

5.0.13 Qualitätsmerkmale von Software

- **Funktionalität:** Die Software erfüllt die festgelegten Anforderungen und Erwartungen.
- **Zuverlässigkeit:** Die Software funktioniert stabil und fehlerfrei.
- **Benutzerfreundlichkeit:** Die Software ist intuitiv und einfach zu bedienen.
- **Wartbarkeit:** Die Software lässt sich leicht anpassen und reparieren.
- **Leistung:** Die Software reagiert schnell und effizient.
- **Sicherheit:** Die Software schützt Daten und verhindert unbefugten Zugriff.

Prozessbezogene und produktbezogene Qualität

- **Prozessbezogene Qualität:** Bezieht sich auf die Qualität der Prozesse, die zur Softwareentwicklung verwendet werden.
- **Produktbezogene Qualität:** Bezieht sich auf die Qualität des Endprodukts selbst.

Warum sind Qualitäten oft schwer messbar?

Qualitäten sind oft schwer messbar, weil:

- Viele Merkmale subjektiv sind.
- Einige Merkmale schwer quantifizierbar sind.
- Keine einheitlichen Standards oder Metriken existieren.

Harte und weiche Kriterien

- **Harte Kriterien:** Objektiv messbare Merkmale.
- **Weiche Kriterien:** Subjektive Merkmale, die schwer zu quantifizieren sind.

Nutzerfreundlichkeit, Wartbarkeit, Fehlertoleranz, Entwicklungsgeschwindigkeit, Entwicklungseffizienz

Diese Merkmale sind entscheidend für die Gesamtqualität einer Softwareanwendung:

- Nutzerfreundlichkeit
- Wartbarkeit
- Fehlertoleranz
- Entwicklungsgeschwindigkeit
- Entwicklungseffizienz

Gibt es rechtliche Rahmenbedingungen bei der Entwicklung von Software?

Ja, es gibt rechtliche Rahmenbedingungen, darunter:

- Urheberrecht und Lizenzierung
- Datenschutzgesetze
- Sicherheitsstandards
- Branchenvorschriften

Woran hat ein Auftraggeber Interesse (**Magisches Dreieck**)?

Ein Auftraggeber hat Interesse an:

- Kosten
- Zeit
- Qualität

Welche Qualität muss das Team aufweisen, damit der Auftraggeber das Projekt vergibt?

Das Team muss über folgende Qualitäten verfügen:

- Fachliche Kompetenz
- Teamarbeit
- Kommunikationsfähigkeiten
- Flexibilität

Warum muss man die Qualitätseigenschaften im Entwurf beachten?

Die Qualitätseigenschaften müssen beachtet werden, weil:

- sie die Grundlage für die Architektur bilden.
- frühe Entscheidungen spätere Probleme verhindern können.
- sie sicherstellen, dass die Software den Anforderungen entspricht.

Ist die Ressourceneffizienz im Entwurf wichtig?

Ja, die Ressourceneffizienz ist wichtig, da sie den Ressourcenverbrauch optimiert und die Gesamtkosten senkt.

Was meint Effizienz?

Effizienz bezieht sich auf das Verhältnis von Ressourcenaufwand zu den erzielten Ergebnissen.

Wann ist ein Produkt "gut" umgesetzt?

Ein Produkt ist "gut" umgesetzt, wenn:

- Es die Anforderungen erfüllt.
- Es zuverlässig und wartbar ist.
- Es positive Rückmeldungen erhält.

(Beispiel Corona-Warn-App)**Unterschied zwischen funktionalen und nicht-funktionalen Anforderungen?**

- Funktionale Anforderungen: Beschreiben, was das System tun soll.
- Nicht-funktionale Anforderungen: Beschreiben, wie das System die funktionalen Anforderungen erfüllen soll.

Was hilft dabei, nicht-funktionale Anforderungen zu testen?

Folgende Methoden helfen dabei:

- Last- und Performancetests
- Sicherheitstests
- Usability-Tests

Welche Probleme kann es beim Erfassen der Merkmale geben?

Probleme können sein:

- Unklare Anforderungen
- Subjektivität
- Mangelnde Kommunikation

Wie kann man **Portabilität sicherstellen?**

Portabilität kann sichergestellt werden durch:

- Verwendung standardisierter Technologien
- Modularer Aufbau
- Dokumentation

Was sind die Vorteile von [Portabilität](#)?

Die Vorteile sind:

- Erleichterte Verbreitung
- Kosteneinsparungen
- Höhere Benutzerfreundlichkeit

Was macht man, wenn der Kunde eine nicht-funktionale Anforderung bemängelt? Wo liegt der Fehler?

Man sollte:

- Die Bedenken klären.
- Die Anforderungen überprüfen.
- Feedback sammeln.

Der Fehler könnte in der unzureichenden Erfassung der Anforderungen liegen.

Was bedeutet es, ein Problem effizient zu lösen?

Ein Problem effizient zu lösen bedeutet, dass die Lösung mit minimalem Aufwand erreicht wird.

Was ist eine nicht-funktionale Anforderung an eine Handy-App?

Eine nicht-funktionale Anforderung könnte die Benutzerfreundlichkeit oder Leistung betreffen.

Welche Qualitätsmerkmale gibt es?

Qualitätsmerkmale können umfassen:

- Funktionalität
- Zuverlässigkeit
- Benutzerfreundlichkeit
- Wartbarkeit
- Leistung
- Sicherheit

Sind nicht-funktionale Merkmale einfacher oder schwerer als funktionale Merkmale zu erfassen und warum?

Nicht-funktionale Merkmale sind oft schwerer zu erfassen, weil sie subjektiv sind und schwer quantifizierbare Eigenschaften betreffen.

5.0.14 Scrum-Team, Schätzungen und [Planning Poker](#)**Was ist das [Planning Poker](#)?**

[Planning Poker](#) ist eine Schätztechnik, die in agilen Projekten, insbesondere in Scrum, verwendet wird, um den Aufwand oder die Komplexität von Aufgaben (insbesondere [User Stories](#)) zu schätzen. Dabei kommen Karten mit verschiedenen Punktwerten zum Einsatz, die von den Teammitgliedern gleichzeitig ausgewählt und aufgedeckt werden, um eine Diskussion über die Schätzungen zu fördern.

Was sind die Kenngrößen?

Die Kenngrößen im [Planning Poker](#) beziehen sich auf die relativen Schätzwerte, die den Teammitgliedern helfen, den Aufwand für eine Aufgabe oder [User Story](#) zu quantifizieren. Oft wird eine Fibonacci-Folge (1, 2, 3, 5, 8, 13, ...) verwendet, um die Schätzungen zu strukturieren und größere Unterschiede in der Komplexität hervorzuheben.

Warum werden Karten verdeckt gelegt?

Karten werden verdeckt gelegt, um die Unabhängigkeit der Schätzungen der Teammitglieder zu gewährleisten. So wird verhindert, dass die Meinungen anderer Teammitglieder die individuellen Schätzungen beeinflussen. Dies fördert eine ehrliche und unvoreingenommene Bewertung des Aufwands.

Warum wird abstrakt geschätzt und nicht in Arbeitsstunden?

Abstrakte Schätzungen (z. B. in **SPs**) werden verwendet, weil sie den Fokus auf die relative Komplexität oder den Aufwand einer Aufgabe legen, anstatt auf eine genaue Zeitschätzung. Dies hilft, Unsicherheiten und Variabilität in der Schätzung zu berücksichtigen und ermöglicht es dem Team, sich auf die relative Größe der Aufgaben zu konzentrieren, anstatt sich auf spezifische Zeitangaben festzulegen.

Warum nicht mit Zahlen von 1 bis 100?

Die Verwendung von Zahlen von 1 bis 100 kann zu einer übermäßigen Genauigkeit führen, die in der frühen Phase der Planung nicht sinnvoll ist. Solche feinen Abstufungen können zu Verwirrung führen und die Diskussion unnötig kompliziert machen. Stattdessen fördert die Verwendung einer begrenzten Skala, wie der Fibonacci-Folge, eine einfachere und klarere Kommunikation über die Schätzungen.

Meint jeder mit 5 Punkten das Gleiche?

Nicht unbedingt. Obwohl alle Teammitglieder ihre Schätzungen auf der gleichen Skala abgeben, kann die Bedeutung von 5 Punkten für verschiedene Personen unterschiedlich sein, abhängig von ihrer Erfahrung, ihrem Wissen über die Aufgabe und ihrer Perspektive. Aus diesem Grund ist es wichtig, nach der Abstimmung Diskussionen zu führen, um die Gründe für die unterschiedlichen Schätzungen zu verstehen und einen gemeinsamen Konsens zu erreichen.

5.0.15 Velocity

Wie ist die Arbeitsgeschwindigkeit eines Teams definiert?

Die **Velocity** wird in der Regel in **SPs** gemessen, die als Maßeinheit für den Aufwand oder die Komplexität von **User Stories** verwendet werden. Am Ende eines **Sprints** wird die Summe der **SPs**, die das Team erfolgreich umgesetzt hat, erfasst. Diese Summe definiert die **Velocity** für diesen **Sprint**.

5.0.16 MVC

Welche Bestandteile hat MVC?

- Model
- View
- Controller

Zeichnen von MVC-Interaktionen

Aktion im View -> View informiert Controller -> Controller verarbeitet Eingabe und aktualisiert Model -> Model ändert Zustand und benachrichtigt View -> View aktualisiert sich

Vorteil(e) des Observer-Pattern (OP)s?

- Entkopplung
- Flexibilität
- Einfache Erweiterbarkeit

Vorteile von MVC?

- Bessere Wartbarkeit
- Wiederverwendbarkeit
- Bessere Testbarkeit
- Flexibilität

Warum nutzt man Architekturmuster?

- Strukturierung
- Wiederverwendbarkeit
- Wartbarkeit
- Testbarkeit
- Skalierbarkeit

Wie sind Model und View gekoppelt?

Gar nicht, das Model sollte keine direkte Abhängigkeit zur View haben. Aber der View ist der Beobachter des Models.

Warum sollte man keine View-Referenzen im Model speichern?

- Verletzung des MVC-Prinzips
- Schlechte Wartbarkeit
- Schwierige Testbarkeit
- Erweiterbarkeit wird erschwert

5.0.17 Prozessmodell in der Software-Entwicklung**Wie sind die Abläufe in IT-Projekten?**

Die Abläufe in IT-Projekten folgen normalerweise einem strukturierten Prozess, der in Phasen unterteilt ist:

- Anforderungsanalyse: Identifizierung und Dokumentation der Anforderungen.
- Entwurf: Planung der Architektur und der Komponenten der Software.
- Entwicklung: Programmierung und Implementierung der Software.
- Testen: Überprüfung der Software auf Fehler.
- Deployment: Bereitstellung der Software für die Benutzer.
- Wartung: Anpassungen und Fehlerbehebungen nach der Auslieferung.

Beispiel anhand des V-Modells?

Das V-Modell zeigt die Phasen der Entwicklung und die entsprechenden Testaktivitäten:

- Anforderungsdefinition
- System- und Software-Design
- Implementierung
- Testen (Komponententest, Integrationstest, Systemtest, Abnahmetest)

Vorgehens- und Prozessmodelle - Unterschiede?

- **Vorgehensmodelle:** Definieren spezifische Schritte und deren Reihenfolge in der Softwareentwicklung (z.B. Wasserfallmodell, agile Methoden).
- **Prozessmodelle:** Umfassen ein breiteres Spektrum, einschließlich Organisation, Verantwortlichkeiten, Dokumentationsstandards und Qualitätssicherungsmaßnahmen.

Was wird im Prozessmodell festgelegt?

Im Prozessmodell werden folgende Aspekte festgelegt:

- Prozessphasen
- Rollen und Verantwortlichkeiten
- Dokumentationsanforderungen
- Qualitätssicherungsmaßnahmen
- Werkzeuge

5.0.18 Testfälle für Fakultätsfunktion

Grenzfälle

Grenzfälle sind spezielle Eingabewerte, die an den Grenzen oder extremen Punkten der Eingabedaten liegen. Für eine Fakultätsfunktion sind folgende Grenzfälle zu berücksichtigen:

Äquivalenzklassen und Beispiele?

Äquivalenzklassen sind Gruppen von Eingabewerten, die das gleiche Verhalten des Systems erwarten lassen. Für eine Fakultätsfunktion können die Äquivalenzklassen wie folgt definiert werden:

Wofür sind Äquivalenzklassen relevant?

Sie helfen, die Anzahl der benötigten Testfälle zu reduzieren, indem Tester sicherstellen, dass alle relevanten Eingabebereiche abgedeckt sind. Durch die Identifizierung von Äquivalenzklassen können Tester effizientere Tests erstellen und sich auf kritische konzentrieren. Diese Methode fördert eine systematische Testfallgenerierung und verbessert die Testabdeckung, während sie gleichzeitig die Effizienz und Effektivität des Testprozesses steigert.

5.0.19 Aufwandsschätzung mit UCPs

Wie berechnet man die UCPs?

Die UCPs (UCPs) werden berechnet, indem man die Komplexität der Use Cases und die Anzahl der Akteure berücksichtigt. Der Prozess umfasst typischerweise die folgenden Schritte:

- Identifikation der Use Cases und Akteure.
- Klassifizierung der Use Cases in einfache, mittlere und komplexe Kategorien.
- Bestimmung der gewichteten Punkte für jeden Use Case.
- Addition der Punkte und Berücksichtigung der Technik- und Umweltfaktoren.

Was ist ein Use Case?

Ein Use Case ist eine Beschreibung einer Interaktion zwischen einem Benutzer (Akteur) und einem System, die darauf abzielt, ein bestimmtes Ziel zu erreichen. Use Cases helfen, die Anforderungen an das System aus der Sicht der Benutzer zu verstehen.

Struktur eines Use Cases

Die Struktur eines Use Cases umfasst typischerweise:

- Titel: Name des Anwendungsfalls.
- Akteure: Benutzer oder Systeme, die mit dem System interagieren.
- Ziel: Was soll erreicht werden?
- Vorbedingungen: Bedingungen, die erfüllt sein müssen, bevor der Anwendungsfall gestartet werden kann.
- Ablauf: Eine Schritt-für-Schritt-Beschreibung der Interaktion.

- **Nachbedingungs:** Zustand des Systems nach der Durchführung des Anwendungsfalls.
- Alternativabläufe: Abweichungen oder Fehlerbehandlungen während des Ablaufs.

Was sind die wichtigsten Punkte eines Use Cases?

Die wichtigsten Punkte eines Use Cases sind:

- Klare Beschreibung des Ziels.
- Identifikation der Akteure und deren Interaktionen.
- Detaillierte Schritt-für-Schritt-Abläufe und Alternativen.
- Akzeptanzkriterien, die die Bedingungen für den Erfolg definieren.

Wie sieht die Formel zur Berechnung der UCPs aus?

Die Formel zur Berechnung der UCPs lautet typischerweise:

$$UCPs = (UUCW + UAW) \times TCF$$

wobei: - UUCW die gewichteten Punkte der einfachen, mittleren und komplexen Use Cases sind.
 - UAW die gewichteten Punkte der Akteure sind.
 - TCF der Technikfaktor ist, der die technischen Anforderungen widerspiegelt.

Wird beschrieben, was passieren soll?

Ja, in einem Use Case wird beschrieben, was passieren soll. Der Use Case definiert die Interaktionen zwischen Akteur und System sowie die erwarteten Ergebnisse.

Was sind Akteure in einem Use Case?

Akteure sind Benutzer oder andere Systeme, die mit dem System interagieren. Sie können direkt (z.B. Endbenutzer) oder indirekt (z.B. ein externes System) sein.

Ist die Anzahl der Akteure in einem Use Case relevant?

Ja, die Anzahl der Akteure in einem Use Case ist relevant, da sie die Komplexität des Use Cases beeinflussen kann. Mehr Akteure können zu komplexeren Interaktionen und Anforderungen führen.

Was sagen Technik- und Umweltfaktoren aus?

Technikfaktoren beziehen sich auf die technischen Anforderungen und Einschränkungen, die die Implementierung beeinflussen, während **Umweltfaktoren** externe Bedingungen beschreiben, die das Projekt betreffen können, wie z.B. gesetzliche Vorschriften, Marktbedingungen oder organisatorische Aspekte.

5.0.20 Äquivalenzklassen

Praktische Bedeutung

Die praktische Bedeutung von Äquivalenzklassen liegt in der Verbesserung der Effizienz von Testprozessen.

Wie kann man sicher sein, dass zwei Testfälle äquivalent sind?

Um sicherzustellen, dass zwei Testfälle äquivalent sind, sollte man:

- Die Eingabewerte analysieren.
- Überprüfen, ob die erwarteten Ergebnisse gleich sind.
- Die Anforderungen und Spezifikationen überprüfen.

Wie finde ich Äquivalenzklassen?

Äquivalenzklassen können gefunden werden durch:

- Analyse der Anforderungen.
- Nutzung von Grenzwertanalyse.
- Diskussion mit Stakeholdern.

Unterschied zu BBTs?

- Äquivalenzklassen sind eine Technik zur Identifizierung von Testfällen.
- BBTs konzentrieren sich auf Eingaben und Ausgaben, ohne Kenntnisse über den internen Code.

(Beispiel Fibonacci-Folge)

Ein Beispiel für Äquivalenzklassen in Bezug auf die Fibonacci-Folge könnte sein:

- Gültige Äquivalenzklasse: Positive ganze Zahlen.
- Ungültige Äquivalenzklasse: Negative Zahlen.

Was kann schiefgehen, wenn man Äquivalenzklassen aufstellt?

Mögliche Probleme sind:

- Übersehen von wichtigen Randfällen.
- Falsches Identifizieren von Äquivalenzklassen.
- Fehlinterpretation der Anforderungen.

Wie weist man starke Äquivalenz nach, wenn der Code vorliegt?

Starke Äquivalenz kann nachgewiesen werden, indem:

- Alle relevanten Eingabewerte getestet werden.
- Die Ergebnisse mit den erwarteten Ausgaben verglichen werden.
- Die Logik des Codes analysiert wird.
- Was bedeutet starke Äquivalenz?
Starke Äquivalenz bedeutet, dass zwei Eingabewerte unter denselben Bedingungen verwendet werden können und das gleiche Ergebnis liefern.

Wie geht man mit Grenzwerten um?

Mit Grenzwerten geht man um, indem man:

- Testfälle für die Grenzwerte erstellt.
- Überprüft, ob das System an diesen Grenzen korrekt funktioniert.

Welche Vermutung hat man bei einer Äquivalenzklasse?

Bei einer Äquivalenzklasse hat man die Vermutung, dass alle Eingabewerte in dieser Klasse das gleiche Verhalten erwarten lassen.

5.0.21 Vorgangsknotennetz

Wie funktioniert die Metra-Potential-Methode (MPM)?

Die MPM funktioniert folgendermaßen:

- **Vorgangsknoten:** Jeder Knoten im Netz stellt einen bestimmten Vorgang dar.
- **Kanten:** Die Kanten zeigen die Abhängigkeiten zwischen den Vorgängen an.
- **Zeitliche Planung:** Analyse des Vorgangsknotennetzes zur Bestimmung der frühesten und spätesten Start- und Endzeiten.

Wie berechnet man **frühester Anfangszeitpunkt (FAP)**, wie **spätester Startzeitpunkt (SSP)**?

- **FAP:**

$$FAP = \max(FEP_{\text{vorgänger}})$$

- **SSP:**

$$\text{spätester Endzeitpunkt}(SEP) = \min(SEP_{\text{nachfolger}}) - \text{Dauer des Vorgangs}$$

Wie können Termine und Zeiten geplant werden?

Termine und Zeiten können geplant werden durch:

- Erstellung eines Vorgangsknotennetzes.
- Bestimmung der Dauer für jeden Vorgang.
- Berechnung der **FAP** und **SSP**.
- Ressourcenplanung zur Sicherstellung der Verfügbarkeit.
- Festlegung von Meilensteinen zur Überwachung des Fortschritts.

5.0.22 Erfolg eines IT-Projekts

Was muss erfüllt sein, damit ein IT-Projekt erfolgreich ist?

Damit ein IT-Projekt erfolgreich ist, sollten folgende Kriterien erfüllt sein:

- Erfüllung der Anforderungen
- Termintreue
- Budgeteinhaltung
- Qualität
- Kundenzufriedenheit

Wie funktioniert gutes Projektmanagement heute?

Gutes Projektmanagement heute umfasst:

- Agile Methoden
- Regelmäßige Kommunikation
- Risikomanagement
- Stakeholder-Engagement
- Nutzung von Projektmanagement-Tools

Was kann man machen, wenn der Zeit- oder Kostenplan knapp wird?

Wenn der Zeit- oder Kostenplan knapp wird, können folgende Maßnahmen ergriffen werden:

- Priorisierung von Anforderungen
- Erhöhung der Ressourcen
- Anpassung des Zeitplans
- Effizienzsteigerungen

Magisches Dreieck

Beschreibt die drei Hauptfaktoren:

- Kosten
- Zeit
- Qualität

Was bedeutet Qualität?

Qualität bezieht sich auf das Maß, in dem ein Produkt oder eine Dienstleistung die festgelegten Anforderungen und Erwartungen erfüllt.

Was sind funktionale und nicht-funktionale Anforderungen?

- **Funktionale Anforderungen:** Beschreiben, was das System tun soll.
- **Nicht-funktionale Anforderungen:** Beschreiben, wie das System die funktionalen Anforderungen erfüllen soll.

Was sind Qualitätseigenschaften?

Qualitätseigenschaften sind Merkmale, die die Qualität eines Softwareprodukts bestimmen, darunter:

- Funktionalität
- Zuverlässigkeit
- Benutzerfreundlichkeit
- Wartbarkeit
- Leistung
- Sicherheit

5.0.23 Entwurfsmuster

Entwurfsmuster sind bewährte Lösungen für häufige Designprobleme in der Softwareentwicklung. Sie können in folgende Kategorien unterteilt werden:

- **Erzeugungsmuster:**
 - Singleton: Stellt sicher, dass eine Klasse genau ein Objekt hat.
 - Factory Method: Definiert eine Schnittstelle zur Erstellung von Objekten.
- **Strukturmuster:**
 - Adapter: Ermöglicht die Zusammenarbeit von Klassen mit inkompatiblen Schnittstellen.
 - Decorator: Fügt einer bestehenden Klasse zusätzliche Verantwortlichkeiten hinzu.
- **Verhaltensmuster:**
 - Observer: Definiert eine Abhängigkeit zwischen Objekten für Benachrichtigungen.
 - Strategy: Ermöglicht die Definition und Änderung von Algorithmen.

Beispiele

Einige praktische Beispiele für **Entwurfsmuster** sind:

- **Singleton:** Eine Konfigurationsklasse, die sicherstellt, dass nur eine Instanz existiert.
- **Factory Method:** Eine Anwendung, die verschiedene Fahrzeugtypen erstellt.
- **Observer:** Ein Benachrichtigungssystem, das Benutzer über Änderungen informiert.

Interfaces

- Flexibilität: Klassen können verschiedene Implementierungen eines **Interfaces** definieren.
- Abstraktion: **Interfaces** bieten eine klar definierte API.
- **Polymorphism** us: Ermöglicht die Verwendung von Objekten unterschiedlicher Klassen über eine Schnittstelle.

5.0.24 Unterschied Architektur- und Entwurfsmuster

- **Architekturmuster:** Beschreiben die grundlegende Struktur und Organisation eines Softwaresystems.
- **Entwurfsmuster:** Konzentrieren sich auf spezifische Probleme innerhalb einer bestimmten Komponenten- oder Modulfunktionalität.

Ist MVC ein Architektur- oder ein Entwurfsmuster?

Das MVC-Muster (Model-View-Controller) ist ein Architekturpattern.

Wie sollten Model und View gekoppelt werden?

Model und View sollten lose gekoppelt sein, um:

- Flexibilität und Wiederverwendbarkeit zu erhöhen.
- Das OP zu verwenden, um Updates zu ermöglichen.
- Interfaces zu nutzen, um Abhängigkeiten zu reduzieren.

Wobei hilft einem ein Architekturmuster?

Ein Architekturmuster hilft bei:

- Strukturierung des Systems.
- Verbesserung der Wartbarkeit.
- Förderung der Wiederverwendbarkeit.
- Standardisierung bewährter Lösungen.

Was wird möglich, wenn es keine Kopplung gibt?

Wenn es keine Kopplung gibt:

- Unabhängigkeit von Komponenten.
- Flexibilität bei Änderungen.
- Erweiterbarkeit ohne Beeinträchtigung bestehender Komponenten.
- Wiederverwendbarkeit in anderen Projekten.

5.0.25 Architekturmuster

Wozu gehören Client-Server-Muster?

Client-Server-Muster gehören zu den Architekturmuster in der Softwareentwicklung. Sie definieren eine Struktur, in der:

- **Client:** Der Teil, der Anfragen an den Server stellt.
- **Server:** Der Teil, der die Anfragen verarbeitet und die erforderlichen Antworten zurücksendet.

Dieses Muster ermöglicht eine verbesserte Skalierbarkeit, Wartbarkeit und Flexibilität.

Was ist das Schichtenmodell?

Das Schichtenmodell ist ein Architekturmuster, das die Software in klar definierte Schichten unterteilt:

- **Präsentationsschicht:** Verantwortlich für die Benutzeroberfläche.
- **Anwendungsschicht:** Beinhaltet die Geschäftslogik.
- **Datenzugriffsschicht:** Verantwortlich für den Zugriff auf Datenspeicher.

Was sind die Vorteile des Schichtenmodells?

Die Vorteile des Schichtenmodells sind:

- Trennung von Anliegen
- Wartbarkeit
- Wiederverwendbarkeit
- Testbarkeit
- Flexibilität

Chapter 6

Anhang

Glossary

Aggregation Eine Beziehung zwischen Klassen in der objektorientierten Programmierung, bei der eine Klasse (das Aggregat) aus einer oder mehreren anderen Klassen (den Komponenten) besteht, aber die Komponenten unabhängig existieren können. Zum Beispiel kann eine Bibliothek eine Aggregation von Buch-Objekten sein, wobei die Bücher auch ohne die Bibliothek existieren können. 28

Architekturmuster Bewährte Lösungen für häufige Probleme in der Softwarearchitektur. Sie definieren die Struktur und Organisation eines Softwaresystems und legen fest, wie die verschiedenen Komponenten interagieren. Beispiele sind das Model-View-Controller (MVC)-Muster, das Schichtenmodell und das Client-Server-Modell. 1, 39, 45

BBT *Black-Box-Test*. Testmethode, bei der der Tester keine Kenntnisse über die interne Implementierung oder den Quellcode des Systems hat. Der Fokus liegt auf der Überprüfung der Funktionalität des Systems durch Eingaben und Ausgaben. Der Tester bewertet, ob das System die erwarteten Ergebnisse basierend auf den gegebenen Eingaben liefert, ohne zu wissen, wie das System intern arbeitet. 1, 31, 32, 42

Client-Server-Muster Ein Architekturmuster, das die Struktur eines Systems beschreibt, in dem Clients Anfragen an einen Server senden, der die Anfragen verarbeitet und die entsprechenden Antworten zurücksendet. Dieses Muster fördert die Trennung von Benutzeroberfläche und Geschäftslogik und ermöglicht Skalierbarkeit und Wartbarkeit. 45

CRC-Karte Eine CRC-Karte (Class-Responsibility-Collaborator-Karte) ist ein Werkzeug zur Analyse und Planung von objektorientierten Klassen. Sie enthält drei Hauptkomponenten: Klasse, Verantwortlichkeiten, Zusammenarbeit. 1, 29, 30

Daily Scrum Kurzes, tägliches Stand-up-Meeting, das im Scrum-Prozess stattfindet. Es dient der Synchronisation des Teams und ermöglicht es den Teammitgliedern, ihren Fortschritt zu teilen, Hindernisse zu identifizieren und die Arbeit für den Tag zu planen. Die Dauer beträgt in der Regel 15 Minuten. 20, 24, 25

Detektor Werkzeug oder eine Komponente in der Softwareentwicklung, die dazu dient, bestimmte Zustände, Bedingungen oder Ereignisse zu erkennen. Detektoren können in verschiedenen Kontexten verwendet werden, beispielsweise zur Identifikation von Fehlern, zur Überwachung der Leistung oder zur Ausführung von Tests. 32

Direkte Kopplung Bezieht sich auf eine Beziehung zwischen Klassen, bei der eine Klasse direkt auf die Methoden oder Attribute einer anderen Klasse zugreift. Diese Art der Kopplung kann zu einer engen Abhängigkeit führen, was die Flexibilität und Wartbarkeit des Systems verringert. 27, 30

Domänenmodell Eine abstrahierte Darstellung der relevanten Konzepte, Beziehungen und Regeln innerhalb eines bestimmten Anwendungsbereichs (Domäne). Es beschreibt die Struktur und das Verhalten der Elemente in diesem Bereich und hilft, die Anforderungen besser zu verstehen und zu kommunizieren. 1, 29, 34

Entscheidungsdiagramm Eine grafische Darstellung von Entscheidungsprozessen, die es ermöglicht, verschiedene Optionen und die daraus resultierenden Konsequenzen zu visualisieren. Es wird häufig verwendet, um die Logik von Programmen zu analysieren und zu dokumentieren. 31

Entscheidungspunkt test. 32

Entwurfsmuster Bewährte Lösungen für wiederkehrende Designprobleme in der Softwareentwicklung. Sie bieten eine allgemeine Vorlage, die in verschiedenen Kontexten angepasst werden kann. Zu den bekanntesten Entwurfsmustern gehören Singleton, Factory Method und Observer. 1, 44, 45

Erzeugungsmuster Eine Kategorie von Entwurfsmustern, die sich mit der Instanziierung von Objekten befassen. Sie bieten Mechanismen zur Erstellung von Objekten auf flexible und wiederverwendbare Weise. Beispiele sind das Singleton-Muster und das Factory-Muster. 44

FAP frühester Anfangszeitpunkt. 43

GBT *Glass-Box-Test*. Testmethode, bei der der Tester Zugriff auf den Quellcode hat und die interne Struktur sowie die Logik des Systems kennt. Diese Art von Test konzentriert sich auf die Überprüfung der Implementierung, indem alle Code-Pfade, Bedingungen und Schleifen getestet werden. Ziel ist es, sicherzustellen, dass der Code korrekt funktioniert und alle möglichen Ausführungspfade abgedeckt sind. 1, 31–33

GRASP-Pattern Spezifische Lösungen oder Entwurfsmuster, die auf den oben genannten Prinzipien basieren. Sie bieten konkrete Ansätze, um wiederkehrende Designprobleme zu lösen. GRASP-Patterns sind oft näher an der praktischen Implementierung und können als Vorlagen für das Design von Klassen und deren Interaktionen verwendet werden. 27, 29, 30

GRASP-Prinzip Grundlegende Leitlinien oder Regeln, die bei der Zuweisung von Verantwortlichkeiten in objektorientierten Designs helfen. Sie bieten allgemeine Empfehlungen, wie man die Struktur und Organisation von Klassen und Objekten gestalten sollte, um die Softwarequalität zu verbessern. 27

High Cohesion Das Prinzip, dass die Methoden und Attribute einer Klasse eng miteinander verbunden sind und eine gemeinsame Verantwortung haben. Eine hohe Kohäsion führt zu besser strukturierten und leichter verständlichen Klassen. 1, 27–31

High Coupling Bezieht sich auf eine starke Abhängigkeit zwischen Klassen. Eine hohe Kopplung kann die Wartbarkeit und Flexibilität eines Systems verringern, da Änderungen in einer Klasse Auswirkungen auf andere Klassen haben können. 27, 28

Indirekte Kopplung Liegt vor, wenn eine Klasse über eine dritte Klasse mit einer anderen Klasse interagiert. Diese Art der Kopplung verringert die direkte Abhängigkeit zwischen Klassen und fördert die Flexibilität. 27, 30

Information Expert GRASP-Pattern, das besagt, dass Verantwortlichkeiten den Klassen zugewiesen werden sollten, die die benötigten Informationen besitzen. Dies fördert die Kohäsion und reduziert die Kopplung. 27, 29

Interface Sammlung von Methoden, die von einer Klasse implementiert werden müssen, ohne dass die Implementierung spezifiziert wird. Interfaces fördern die lose Kopplung und ermöglichen Polymorphismus, indem sie eine gemeinsame Schnittstelle für verschiedene Klassen bereitstellen. 27, 28, 44, 45

Kohäsion Bezieht sich auf das Maß, in dem die Elemente einer Klasse (Methoden und Attribute) zusammengehören. Hohe Kohäsion bedeutet, dass die Elemente eng miteinander verbunden sind und eine klare, konsistente Aufgabe erfüllen, was die Verständlichkeit und Wartbarkeit der Klasse erhöht. 27–29

Kontrollflussdiagramm Eine grafische Darstellung des Ablaufs eines Programms oder eines Algorithmus. Es zeigt die verschiedenen Schritte, Entscheidungen und Abläufe in einem Prozess. Kontrollflussdiagramme helfen dabei, die Logik und Struktur eines Codes zu visualisieren und zu analysieren. 31, 32

Kopplung Beschreibt die Abhängigkeit zwischen Klassen oder Modulen in einem Software-System. Hohe Kopplung bedeutet, dass Klassen stark voneinander abhängig sind, während niedrige Kopplung darauf hinweist, dass Klassen weitgehend unabhängig sind. Niedrige Kopplung ist wünschenswert, da sie die Wartbarkeit und Flexibilität des Systems erhöht. 1, 27–30, 45

Lastenheft Dokument, das die Anforderungen und Erwartungen des Auftraggebers an ein System beschreibt. Es definiert, was das System tun soll, ohne zu spezifizieren, wie dies erreicht werden soll. Das Lastenheft dient als Grundlage für die weitere Planung und Entwicklung. 21, 23, 31

Low Coupling Prinzip, das darauf abzielt, die Abhängigkeiten zwischen Klassen zu minimieren. Eine niedrige Kopplung fördert die Unabhängigkeit der Klassen, was die Wartbarkeit, Testbarkeit und Wiederverwendbarkeit des Codes verbessert. 27–31

Magisches Dreieck Kosten - Zeit - Qualität. 35, 43

MPM *Metra-Potential-Methode*. Technik zur Planung von Projekten, die auf einem Vorgangsknotennetz basiert. Sie dient dazu, die Abhängigkeiten zwischen den Aufgaben zu visualisieren und die erforderlichen Ressourcen und Zeitpläne zu optimieren. Die Methode hilft, früheste und späteste Start- sowie Endzeiten für Vorgänge zu berechnen. 42

MVC Model-View-Controller. 1, 38, 39, 45

Nachbedingung Definiert den Zustand, der nach der Ausführung eines Anwendungsfalls oder einer Funktion erreicht werden sollte. Sie beschreibt, was im System nach der Durchführung einer Aktion oder eines Vorgangs erwartet wird und ist wichtig für die Validierung der Funktionalität. 22, 24, 41

OP *Observer-Pattern*. Verhaltensmuster, das eine Eins-zu-viele-Abhängigkeit zwischen Objekten definiert, sodass, wenn sich der Zustand eines Objekts ändert, alle abhängigen Objekte (Beobachter) automatisch benachrichtigt und aktualisiert werden. Dieses Muster wird häufig in Event-Driven-Architekturen eingesetzt.. 38, 45

PBI *Product-Backlog-Item*. Anforderungen, die im Product Backlog eines Projekts festgehalten werden. 9, 10

Pfad Vollständige Sequenz von Anweisungen, die während der Ausführung eines Programms durchlaufen wird. Pfade sind wichtig für die Analyse des Kontrollflusses und die Erstellung von Testfällen, um sicherzustellen, dass alle Codepfade abgedeckt werden. 31–33

Pfadüberdeckung Ein Maß dafür, inwieweit die verschiedenen Ausführungspfade in einem Programm getestet werden. Eine 100%-ige Pfadüberdeckung bedeutet, dass alle möglichen Pfade durch das Programm ausgeführt wurden. Dies ist ein wichtiger Aspekt der Testabdeckung. 32, 33

Pflichtenheft Dokument, das die detaillierten Anforderungen an ein System beschreibt, einschließlich technischer Spezifikationen und Implementierungsdetails. Es wird in der Regel auf der Grundlage des Lastenhefts erstellt und dient als Grundlage für die Entwicklung. 21, 23, 31

Planning Poker test. 1, 19, 22, 37

PO *Product Owner*. Rolle im Scrum-Framework, die für das Management des Product Backlogs verantwortlich ist. Der Product Owner priorisiert die Anforderungen, stellt sicher, dass das Team die richtigen Funktionen umsetzt, und fungiert als Schnittstelle zwischen den Stakeholdern und dem Entwicklungsteam. 9–11, 20, 24

Polymorphism Konzept in der objektorientierten Programmierung, das es ermöglicht, dass verschiedene Klassen auf die gleiche Weise behandelt werden können, indem sie eine gemeinsame Schnittstelle oder Basisklasse verwenden. Es fördert die Flexibilität und Wiederverwendbarkeit des Codes. 29, 30, 44

Portabilität Fähigkeit einer Software, auf verschiedenen Plattformen oder in unterschiedlichen Umgebungen ohne wesentliche Anpassungen zu laufen. Hohe Portabilität ermöglicht es, die Software effizient zu verteilen und auf verschiedenen Systemen zu verwenden. 36, 37

Product Backlog Priorisierte Liste von Anforderungen, Features und Verbesserungen für ein Produkt, die vom Product Owner verwaltet wird. Es dient als zentrales Dokument für die Planung und Durchführung der Sprints. 20, 24

Prototyping Prozess der Erstellung von Modellen oder Mockups einer Softwareanwendung, um Anforderungen zu testen und Feedback von Benutzern zu sammeln. Prototypen helfen, die Benutzererfahrung zu visualisieren und sicherzustellen, dass die Anforderungen korrekt verstanden werden. 23

Pure Fabrication GRASP-Pattern, das vorschlägt, eine Klasse zu erstellen, die nicht direkt aus den Anforderungen oder dem Domänenmodell abgeleitet ist, um die Kopplung zu reduzieren und die Kohäsion zu erhöhen. Diese Klasse wird verwendet, um spezifische Verantwortlichkeiten zu kapseln. 29, 30

Referenz-Item Beispiel oder eine Vorlage aus einem bestehenden System oder Projekt sein, das zur Unterstützung bei der Entwicklung neuer Anforderungen oder Designs verwendet wird. Es dient als Leitfaden oder Inspirationsquelle. 10

Schichtenmodell Architekturmuster, das Software in mehrere Schichten unterteilt, wobei jede Schicht eine bestimmte Verantwortung hat. Typische Schichten sind die Präsentationsschicht, die Anwendungsschicht und die Datenzugriffsschicht. Es fördert die Trennung von Anliegen und erleichtert die Wartung und Erweiterung der Software. 45, 46

semantische Lücke Tritt auf, wenn es einen Unterschied zwischen den Erwartungen der Stakeholder und dem tatsächlichen Verhalten eines Systems gibt. Sie beschreibt die Diskrepanz zwischen dem, was Benutzer oder Stakeholder von einem System erwarten und dem, was das System tatsächlich liefert. 34

SEP spätester Endzeitpunkt. 43

SM Scrum Master. 9–11, 24

SP Story Point. 20–22, 26, 38

SPL Selected Product Backlog. 9, 10

Sprint Festgelegte, zeitlich begrenzte Phase im Scrum-Prozess, in der ein funktionsfähiges Produktinkrement entwickelt wird. Sprints dauern in der Regel zwischen 1 und 4 Wochen. Am Ende eines Sprints wird das entwickelte Produkt überprüft und bewertet. 19–21, 24–26, 38

Sprint Backlog Liste von User Stories, Aufgaben und Anforderungen, die das Team während eines Sprints umsetzen möchte. Es wird zu Beginn des Sprints im Sprint Planning Meeting erstellt und stellt sicher, dass das Team klare Ziele hat. 10, 21, 24

Sprint Execution Die Phase, in der das Entwicklungsteam die im Sprint Backlog festgelegten Aufgaben umsetzt. Diese Phase umfasst die eigentliche Entwicklungsarbeit, das Testen und die Integration der Software. 24

Sprint Planning Meeting, das zu Beginn jedes Sprints stattfindet. In diesem Meeting bestimmt das Team, welche User Stories aus dem Product Backlog für den nächsten Sprint ausgewählt werden und plant, wie die Arbeit ausgeführt wird. 19, 24–26

Sprint Retrospective Meeting, das am Ende eines Sprints abgehalten wird, in dem das Team darüber reflektiert, was gut gelaufen ist, was verbessert werden kann und welche Maßnahmen für den nächsten Sprint ergriffen werden sollen. 20, 24, 25

Sprint Review Meeting, das am Ende eines Sprints stattfindet, in dem das Team das entwickelte Produktinkrement präsentiert. Stakeholder geben Feedback, und das Team diskutiert, ob die Ziele des Sprints erreicht wurden. 19–21, 24, 25

SSP spätester Startzeitpunkt. 43

Strukturmuster Kategorie von Entwurfsmustern, die sich mit der Zusammensetzung von Klassen und Objekten befassen. Sie helfen, die Beziehungen zwischen Objekten zu organisieren und die Struktur des Softwaredesigns zu verbessern. Beispiele sind das Adapter-Muster und das Decorator-Muster. 44

Szenario Spezifische Beschreibung einer Interaktion zwischen einem Benutzer (Akteur) und einem System, die eine bestimmte Situation oder Anforderung darstellt. Szenarien werden häufig in der Anforderungsanalyse verwendet, um die Benutzererfahrung zu veranschaulichen. 24

Technikfaktor Parameter, der in der Aufwandsschätzung, insbesondere bei Use Case Points, berücksichtigt wird. Er reflektiert die technischen Herausforderungen und Anforderungen, die das Projekt beeinflussen können, und hilft, die geschätzte Komplexität zu bewerten. 41

Termbaum Grafische Darstellung von Ausdrücken oder Berechnungen in der Informatik. Er zeigt die Struktur von Ausdrücken und deren Auswertungsreihenfolge und wird häufig in der Programmierung und im Compilerbau verwendet. 33

UCP *Use Case Points*. Methode zur Aufwandsschätzung in der Softwareentwicklung, die auf der Analyse von Use Cases basiert. UCP berücksichtigt die Anzahl und Komplexität der Use Cases sowie die Anzahl der Akteure, um den Aufwand für die Entwicklung zu schätzen. 1, 40, 41

Umweltfaktor Parameter, der in der Aufwandsschätzung berücksichtigt wird und externe Bedingungen beschreibt, die das Projekt beeinflussen können. Dazu gehören beispielsweise Marktbedingungen, organisatorische Rahmenbedingungen und gesetzliche Vorgaben. 40, 41

- USC** *User Story Card*. Dokument, das eine User Story beschreibt. Sie enthält typischerweise den Titel der Story, eine Beschreibung aus der Sicht des Benutzers und Akzeptanzkriterien, die definieren, wann die Story als abgeschlossen gilt. 21
- Use Case** Beschreibung einer Interaktion zwischen einem Benutzer (Akteur) und einem System, um ein bestimmtes Ziel zu erreichen. Use Cases helfen dabei, die Anforderungen an das System aus der Perspektive der Benutzer zu verstehen. 21, 22, 24, 29–31, 40, 41
- User Story** Kurze, einfache Beschreibung einer Anforderung aus der Sicht des Endbenutzers. Sie folgt typischerweise dem Format: "Als [Benutzertyp] möchte ich [Ziel], damit [Nutzen]." User Stories sind ein zentrales Element agiler Methoden. 19–24, 26, 37, 38
- Velocity** Maß für die Menge an Arbeit, die ein Scrum-Team in einem Sprint erledigen kann. Sie wird in der Regel in Story Points gemessen und hilft dem Team, seine Kapazität für zukünftige Sprints besser zu schätzen.. 1, 10, 20, 26, 38
- Verhaltensmuster** Kategorie von Entwurfsmustern, die sich mit der Interaktion zwischen Objekten und der Kommunikation zwischen ihnen befassen. Beispiele sind das Observer-Muster und das Strategy-Muster. 44
- Zweig** Entscheidungspunkt im Code, an dem der Fluss basierend auf einer Bedingung in verschiedene Richtungen verzweigt. Zum Beispiel in einer if-Anweisung, wo verschiedene Ausführungspfade existieren. 32, 33
- Zweigüberdeckung** Testmetrik, die misst, ob alle Entscheidungspunkte im Code getestet wurden. Sie stellt sicher, dass alle möglichen Wege durch die Bedingungen im Code abgedeckt sind. 32, 33
- Äquivalenzklasse** Eine Äquivalenzklasse ist eine Gruppe von Eingabewerten, die das gleiche Verhalten eines Systems erwarten lassen.. 1, 31, 33, 40–42